INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

ESCOM

Trabajo Terminal

"PROTOTIPO DE UN SISTEMA WEB DE ANÁLISIS CONDUCTUAL ASISTIDO POR IA EN UN MODELO DE CONDUCTA DEPRESIVA EN RATA MEDIANTE NADO FORZADO"

2026-B066

Presentan

Frausto Robles Angel Ali

Rodriguez Verdin Sandoval Vanesa

Directores

Cordero Lopez Martha Rosa

Salas Ramírez Israel

Mayo 2026

Índice general

1. Introducción **1**
1.1. Antecedentes . . . 1
1.2. Planteamiento del problema . . . 2
1.3. Propuesta de solución . . . 3
1.4. Objetivo general . . . 4
1.4.1. Objetivos específicos . . . 4
1.5. Justificación . . . 4
1.6. Alcance del proyecto . . . 5
1.7. Metodología . . . 6

2. Estado del arte **9**
2.1. Limitaciones comunes y contexto del proyecto . . . 9
2.2. Trabajos similares . . . 10
2.2.1. DSAAN: clasificación sin umbral en el FST . . . 10
2.2.2. Red 3D-RCNN para clasificación de tres conductas . . . 10
2.2.3. DLC como clasificador de conducta en el FST . . . 10
2.2.4. YOLO-Behaviour: detección y clasificación en dos etapas . . . 11
2.3. Herramientas comerciales de análisis conductual por video . . . 11
2.3.1. EthoVision XT . . . 11
2.3.2. ANY-maze . . . 12
2.4. Herramientas de código abierto para análisis del FST . . . 12
2.4.1. ezTrack . . . 12
2.4.2. DBscorer . . . 12
2.5. Técnicas de visión por computadora para seguimiento de animales . . . 13
2.5.1. Sustracción de fondo . . . 13
2.5.2. YOLO (You Only Look Once) . . . 13
2.5.3. DeepLabCut . . . 14
2.5.4. Trackers de correlación: CSRT y KCF . . . 14
2.6. Modelos de clasificación supervisada aplicables al FST . . . 14

2.6.1. Clasificadores clásicos sobre descriptores de movimiento 14
2.6.2. Redes convolucionales con transfer learning 15
2.6.3. Redes 3D para información temporal 15
2.7. Criterios de calidad para la evaluación del sistema 16
2.8. Tabla comparativa 16

3. Marco teórico 18
3.1. El modelo de nado forzado como paradigma preclínico 18
3.1.1. Depresión y modelos animales 18
3.1.2. La prueba de nado forzado (FST) 19
3.1.3. Conductas que se observan y cómo se interpretan 19
3.1.4. Protocolo del laboratorio colaborador 19
3.2. Procesamiento de video para detección y seguimiento 20
3.2.1. Representación digital de imágenes y video 20
3.2.2. Modelo de fondo y sustracción 20
3.2.3. Umbralización de Otsu 20
3.2.4. Detección de contornos e IoU 21
3.2.5. Estimación de la línea de agua 21
3.2.6. Trackers de correlación: CSRT y KCF 21
3.3. Redes neuronales y aprendizaje profundo 22
3.3.1. Redes neuronales y CNN 22
3.3.2. Transfer learning 22
3.3.3. ResNet y conexiones residuales 23
3.3.4. Clasificación de las tres conductas 23
3.4. Arquitectura del sistema web 24
3.4.1. Arquitectura cliente-servidor y API REST 24
3.4.2. Procesamiento asíncrono con colas de tareas 24
3.4.3. Contenedorización con Docker 24
3.4.4. Persistencia de datos con PostgreSQL 25
3.5. Métricas de evaluación 25
3.5.1. Kappa de Cohen 25
3.5.2. Precisión, recall y F1 por clase 25
3.5.3. Error absoluto medio (MAE) 26
3.5.4. Tiempo de análisis 26

4. Análisis **27**
4.1. Requerimientos . . . 27
4.1.1. Requerimientos Funcionales (RF) . . . 27
4.1.2. Requerimientos No Funcionales (RNF) . . . 33
4.1.3. Reglas de negocio . . . 36
4.2. Análisis de las herramientas a utilizar . . . 38
4.2.1. Detección de objetos y seguimiento . . . 38
4.2.2. Aprendizaje profundo para clasificación de conductas . . . 39
4.2.3. Desarrollo web e infraestructura . . . 40
4.3. Análisis de riesgos . . . 41
4.4. Análisis de factibilidad . . . 44
4.4.1. Factibilidad tecnológica . . . 44
4.4.2. Factibilidad operativa . . . 45
4.4.3. Factibilidad económica . . . 46
4.4.4. Factibilidad legal . . . 47
4.5. Análisis de sostenibilidad . . . 47
4.5.1. Sostenibilidad ambiental . . . 47
4.5.2. Sostenibilidad social e institucional . . . 48
4.5.3. Mantenibilidad del sistema . . . 48

5. Diseño **49**
5.1. Arquitectura general del sistema . . . 49
5.2. Diseño del pipeline de análisis conductual . . . 50
5.2.1. Preprocesamiento . . . 50
5.2.2. Detección de cilindros . . . 51
5.2.3. Seguimiento de animales . . . 51
5.2.4. Clasificación de conducta . . . 51
5.2.5. Estrategia de separación por sujeto . . . 52
5.3. Diseño de la base de datos . . . 52
5.4. Diseño de la API REST . . . 53
5.5. Diseño de casos de uso . . . 55
5.6. Diagramas de secuencia . . . 56
5.7. Diagrama de clases . . . 56
5.8. Diseño de la interfaz web . . . 57
5.8.1. Pantalla de inicio de sesión . . . 57

5.8.2. Dashboard del investigador 57
5.8.3. Creación de experimento 57
5.8.4. Progreso del análisis 58
5.8.5. Resultados del experimento 58
5.8.6. Panel de administración 59

Índice de figuras

5.1. Arquitectura general del sistema [autoría propia]. 49
5.2. Diagrama de flujo del pipeline de análisis conductual [autoría propia]. . . . 50
5.3. Arquitectura del clasificador de conducta basado en ResNet-18 con transfer learning [autoría propia]. 52
5.4. Modelo entidad-relación físico del sistema [autoría propia]. 52
5.5. Diagrama de casos de uso del sistema [autoría propia]. 55
5.6. Secuencia: inicio de sesión [autoría propia]. 56
5.7. Secuencia: carga de video y análisis automático [autoría propia]. 56
5.8. Secuencia: manejo de error en el análisis [autoría propia]. 56
5.9. Diagrama de clases del sistema [autoría propia]. 57
5.10. Pantalla de inicio de sesión [autoría propia]. 57
5.11. Dashboard del investigador [autoría propia]. 57
5.12. Pantalla de creación de experimento [autoría propia]. 58
5.13. Pantalla de progreso del análisis [autoría propia]. 58
5.14. Pantalla de resultados del experimento [autoría propia]. 58
5.15. Panel de administración [autoría propia]. 59

Índice de cuadros

1.1. Plan de trabajo por etapa académica. 8

2.1. Comparativa de herramientas y trabajos relacionados con el análisis del FST. 17

4.1. Requerimientos Funcionales (RF). 28
4.2. Requerimientos No Funcionales (RNF). 33
4.3. Reglas de negocio. 36
4.4. Técnicas de detección de objetos y seguimiento. 39
4.5. Arquitecturas de aprendizaje profundo para clasificación de conductas. . . 40
4.6. Herramientas de desarrollo web e infraestructura. 41
4.7. Matriz de riesgos del proyecto. 42
4.8. Análisis de costos del proyecto. 46

5.1. Endpoints principales de la API REST. 53

Capítulo 1

Introducción

1.1. Antecedentes

El análisis del comportamiento animal con herramientas computacionales cambió bastante en la última década. Lo que antes requería que alguien se sentara horas frente a un video con cronómetro en mano ahora puede delegarse a sistemas que procesan miles de cuadros por minuto, con resultados más consistentes y menos dependientes del criterio del observador [1], [2].

En farmacología experimental, los roedores —especialmente ratas y ratones— son el modelo biológico más usado para estudiar trastornos del estado de ánimo y probar nuevos fármacos. Su fisiología y neuroanatomía tienen correspondencia suficiente con la humana, en particular en la organización del sistema nervioso central y las respuestas neuroendocrinas al estrés [1]. A eso se suman razones prácticas: manejo sencillo, ciclos reproductivos cortos y bajo costo de mantenimiento.

La depresión mayor afecta a más de 280 millones de personas y es una de las principales causas de discapacidad según la OMS [3]. Para probar nuevos antidepresivos antes de pasar a ensayos clínicos en humanos, los investigadores usan modelos conductuales que reproducen algunos aspectos del trastorno. El más citado en neurociencia preclínica es la prueba de nado forzado (Forced Swim Test, FST), propuesta por Porsolt et al. en 1977 [4]. Desde el año 2000 hay más de 8,000 artículos que lo reportan [2].

El FST consiste en poner a la rata en un cilindro con agua del que no puede salir y registrar cómo responde. Al principio intenta escapar; después de un rato empieza a flotar con movimientos mínimos. Esa conducta pasiva —llamada *desesperanza conductual* o *behavioral despair*— se asocia con la pérdida de motivación propia de la depresión [1].

Los antidepresivos reducen el tiempo de inmovilidad, y eso es lo que se mide [5], [6].

El uso de animales de laboratorio en México está regulado por la Norma Oficial Mexicana NOM-062-ZOO-1999 [7], que establece las especificaciones éticas, sanitarias y de bienestar animal para su cuidado y uso experimental. El laboratorio colaborador opera bajo esas disposiciones; este proyecto solo analiza los videos que resultan de los experimentos, sin intervenir en ningún procedimiento con los animales.

Existen herramientas para automatizar parte del análisis. EthoVision XT [8] y ANY-maze son las comerciales más usadas; ezTrack [9] y DBscorer [10] son las alternativas abiertas más específicas para el FST. Todas tienen el mismo problema de fondo: no distinguen las tres conductas relevantes del FST (nado, inmovilidad y escalamiento) por separado, y ninguna tiene interfaz web. Los trabajos más recientes con aprendizaje profundo [2], [11] sí alcanzan esa clasificación de tres clases, pero no están disponibles como sistemas que un laboratorio sin infraestructura especializada pueda desplegar.

El Laboratorio de Neurociencia Conductual de la ENMyH-IPN, a cargo del Dr. César Augusto Sandino Reyes López, investigador del posgrado de esa institución, corre experimentos de FST regularmente y tiene un acervo de videos que hoy se analizan a mano. Esa es la necesidad que origina este Trabajo Terminal.

1.2. Planteamiento del problema

El laboratorio realiza experimentos de nado forzado de forma periódica para evaluar el efecto de distintas moléculas en grupos de ratas. Cada experimento genera videos donde se graban cuatro animales al mismo tiempo desde una vista cenital, con teléfono o tableta, en dos sesiones: 20 minutos el primer día y 5 minutos el segundo.

El análisis se hace completamente a mano: un investigador reproduce la grabación y va anotando con un cronómetro cuándo cada rata nada, cuándo está inmóvil y cuándo intenta escalar. Los problemas son:

- **Tiempo:** analizar un experimento completo puede llevar varias horas, según el número de animales y sesiones.
- **Variabilidad entre evaluadores:** dos personas pueden usar criterios distintos para decidir cuándo empieza o termina una conducta, lo que afecta la reproducibilidad [1]. La norma ISO/IEC 25010:2023 [12] define la consistencia de resultados entre evaluaciones como un atributo de calidad funcional (*correctness*) que un sistema automatizado puede garantizar donde el proceso manual no puede.

- **Sin registro centralizado:** los resultados se guardan en archivos separados —hojas de cálculo, notas— sin forma fácil de comparar sesiones o grupos entre sí.
- **Análisis temporal limitado:** un desglose minuto a minuto sería útil para ver la dinámica del efecto farmacológico, pero hacerlo a mano es prácticamente inviable.

Eso limita cuántos experimentos puede procesar el laboratorio y complica estandarizar los resultados entre sesiones o con otros grupos.

1.3. Propuesta de solución

La propuesta es un prototipo de sistema web con inteligencia artificial que analice automáticamente los videos del FST del laboratorio. El desarrollo sigue el ciclo de vida de software de la norma ISO/IEC/IEEE 12207:2017 [13], que organiza el proceso en fases de definición de requerimientos, diseño, implementación y verificación. La adaptación concreta de ese ciclo al proyecto se describe en la sección de metodología.

El sistema tiene dos partes:

- **Pipeline de análisis de video:** recibe el video, identifica a cada rata dentro de su cilindro y clasifica cuadro a cuadro la conducta que realiza (nado, inmovilidad o escalamiento) con un modelo de aprendizaje supervisado. La salida es el tiempo total por conducta y animal y, como funcionalidad opcional, el desglose por minuto.
- **Plataforma web:** interfaz desde el navegador donde el investigador sube los videos, consulta el estado del análisis, revisa resultados por animal y sesión, y descarga reportes. Los datos se almacenan en base de datos para comparar experimentos.

El sistema asume calidad mínima de entrada: vista cenital con los cuatro cilindros visibles, contraste suficiente entre el animal y el fondo, y cámara estable. Los videos disponibles hasta ahora tienen variaciones de iluminación y contraste que pueden afectar el desempeño; durante el desarrollo se evaluará la robustez de los algoritmos y se documentarán los requisitos mínimos de calidad conforme al atributo *performance efficiency* de la ISO/IEC 25010:2023 [12].

La validación compara las predicciones del sistema con los registros conductuales manuales del laboratorio (gold standard), usando el coeficiente kappa de Cohen y el error absoluto medio por animal.

1.4. Objetivo general

Diseñar, desarrollar y validar un prototipo de sistema web asistido por inteligencia artificial que automatice la detección y cuantificación de las conductas de la prueba de nado forzado en ratas —nado activo, inmovilidad e intentos de escape— con resultados comparables a la evaluación manual del laboratorio y cumpliendo los atributos de calidad de la norma ISO/IEC 25010:2023 [12].

1.4.1. Objetivos específicos

1. Revisar trabajos y herramientas existentes para el análisis automatizado del FST e identificar los enfoques de visión por computadora y clasificación supervisada más adecuados al problema.

2. Documentar el protocolo experimental del laboratorio: qué conductas se anotan, cómo se graba, y qué características tienen los videos disponibles. Esto requiere entrevistas con el equipo del Dr. Sandino y revisión de los registros conductuales que éste proporcionará.

3. Diseñar la arquitectura del sistema conforme al proceso de diseño de software de la ISO/IEC/IEEE 12207:2017 [13], definiendo los módulos de carga de videos, pipeline de análisis, base de datos y visualización de resultados.

4. Implementar el módulo de análisis conductual con visión por computadora y un clasificador supervisado para las tres conductas del FST.

5. Validar el desempeño del sistema comparando sus predicciones contra los registros conductuales del laboratorio (gold standard), usando el coeficiente kappa de Cohen (κ) y el error absoluto medio como métricas principales.

6. Cuantificar la reducción de tiempo de análisis respecto al proceso manual, y dejar documentado qué tendría que cambiar en el pipeline si el laboratorio quisiera aplicarlo a otro paradigma conductual.

1.5. Justificación

El análisis manual del FST tiene tres problemas: toma mucho tiempo, depende del criterio del evaluador y no hay forma centralizada de guardar y comparar los resultados.

Un sistema automatizado resuelve los tres.

Las herramientas comerciales (EthoVision XT, ANY-maze) no están diseñadas para el flujo de trabajo del laboratorio y tienen costos de licencia fuera del alcance académico del IPN. Las abiertas (ezTrack, DBscorer) no clasifican las tres conductas del FST ni tienen plataforma web. Los trabajos con aprendizaje profundo que sí distinguen las tres conductas [2], [11] no se distribuyen como sistemas desplegables. Este prototipo atiende esas tres brechas.

En cuanto a calidad del software, el sistema se diseña para cumplir con atributos concretos de la ISO/IEC 25010:2023 [12]: *functional correctness* (resultados consistentes con las anotaciones del laboratorio), *time behaviour* (el análisis termina en un tiempo que el laboratorio puede asumir en su flujo de trabajo) y *usability* (un investigador sin conocimientos de programación puede operar el sistema sin ayuda técnica). Esos tres atributos son los criterios de aceptación de la validación final.

1.6. Alcance del proyecto

Incluido en el alcance

- Análisis de videos del FST del laboratorio de la ENMyH-IPN, grabados desde arriba con cuatro cilindros simultáneos, bajo condiciones de grabación suficientes para que el procesamiento sea viable.
- Clasificación de tres conductas: nado activo, inmovilidad e intentos de escape. Si el sistema no logra distinguir con precisión suficiente nado y escalamiento, ambos se reportarán juntos como conducta activa.
- Métricas por animal y por sesión: tiempo total de cada conducta en los 5 minutos del día 2 y desglose por minuto como funcionalidad opcional.
- Comparación entre sesiones día 1 y día 2 para el mismo animal o grupo.
- Plataforma web con carga de videos, consulta de resultados y generación de reportes en PDF y CSV.
- Validación contra el gold standard del laboratorio con κ de Cohen y error absoluto medio.

- Documentación técnica conforme a los lineamientos de la CATT-ESCOM y al ciclo de vida de software de la ISO/IEC/IEEE 12207:2017 [13].

Fuera del alcance

- Análisis en tiempo real sobre video en vivo.
- Corrección automática de videos con mala iluminación, reflejos excesivos u otros problemas de calidad de grabación.
- Cualquier intervención en los experimentos con animales; eso lo hace el laboratorio bajo la NOM-062-ZOO-1999 [7].
- Despliegue institucional definitivo; el producto entregado es un prototipo funcional.
- Análisis de paradigmas conductuales distintos al FST en esta versión.

1.7. Metodología

El proyecto usa una metodología ágil basada en Scrum, adaptada a un equipo de dos personas con entregas en dos etapas académicas (TT1 y TT2). El motivo de elegir este enfoque es directo: el proyecto tiene partes muy distintas —el módulo de video, el clasificador, la plataforma web, la base de datos— y no tiene sentido diseñar todo desde el inicio sin saber cómo va a quedar el módulo de seguimiento hasta probarlo. Scrum permite ajustar decisiones al cierre de cada bloque, con base en lo que funcionó.

Cada iteración termina con un entregable revisable con los directores en sesiones quincenales. Las actividades se organizan según los procesos de implementación y verificación de la ISO/IEC/IEEE 12207:2017 [13]: criterios de aceptación definidos antes de codificar, revisión del producto al cierre de cada bloque, y registro de ajustes para el siguiente ciclo.

La Tabla 1.1 muestra la distribución de actividades por etapa y por integrante. Ángel Ali es responsable del sistema web e infraestructura (backend, base de datos, frontend); Vanesa es responsable del pipeline de IA (preprocesamiento de video, clasificador de conducta, validación).

El stack técnico cubre los dos frentes del proyecto: Python con Flask, OpenCV, TensorFlow/scikit-learn y PostgreSQL para el pipeline y el backend; React.js para el frontend; BORIS para anotar el gold standard; Git con Docker para control de versiones y despliegue.

En la práctica, Ángel y Vanesa trabajan en ramas separadas del repositorio y se sincronizan en los sprint reviews semanales. La integración entre los dos módulos —el endpoint que recibe el video y el pipeline que lo procesa— se prueba de forma conjunta al cierre de cada sprint, no se deja acumular para el final.

Cuadro 1.1: Plan de trabajo por etapa académica.

Etapa	Bloque	Ángel Ali (sistema web)	Vanesa (pipeline IA)
TT1 Feb–Jun	Documentación caps. 1–2	Introducción, planteamiento, estado del arte y comparativa tecnológica.	Antecedentes FST, técnicas de visión por computadora y modelos ML/DL aplicables.
	Análisis (cap. 3)	RF/RNF, casos de uso, matriz de trazabilidad y criterios de aceptación por módulo.	Especificación de conductas, métricas, protocolo de anotación y plan de validación.
	Diseño (cap. 4)	Arquitectura del sistema, modelo E-R, API REST, diagramas UML y wireframes.	Pipeline de preprocesamiento, separación por sujeto, descriptores y arquitectura del clasificador.
	Entregable TT1	Protocolo registrado en CATT-ESCOM; documentación de caps. 1–4 aprobada por directores.	
TT2 Jul–Ene	Implementación base	Backend Flask + PostgreSQL, esquema de BD, API REST de sesiones y autenticación.	Baseline de sustracción de fondo, curación de videos y anotación del gold standard con BORIS.
	Implementación principal	Vistas React.js (carga, dashboard, reportes) e integración frontend-backend.	Clasificador entrenado (nado/inmovilidad/escape), ajuste de hiperparámetros y métricas por individuo.
	Integración, pruebas y robustez	Integración del módulo de IA al backend (worker/cola), pruebas unitarias y de sistema.	Pruebas con distintas condiciones de grabación, manejo de oclusiones y reporte de métricas finales.
	Entregable TT2	Sistema desplegado con Docker Compose; manuales técnico y de usuario; material de defensa.	

Capítulo 2

Estado del arte

Antes de definir cómo va a funcionar el sistema, revisamos qué herramientas y trabajos existen para el análisis automatizado del FST. La búsqueda se organizó en cuatro áreas: trabajos similares directamente relacionados con el problema, software comercial y de código abierto para análisis conductual por video, técnicas de visión por computadora y aprendizaje profundo aplicadas al seguimiento de animales, y enfoques de clasificación de conducta con modelos supervisados.

2.1. Limitaciones comunes y contexto del proyecto

Todos los trabajos revisados se validaron bajo condiciones controladas: iluminación uniforme, fondo contrastante, sin reflejos problemáticos. Las herramientas basadas en umbralización (DBscorer, ezTrack) son especialmente sensibles a eso. Los métodos de aprendizaje profundo son más robustos en principio, pero dependen de que el conjunto de entrenamiento represente las condiciones reales de uso.

Los videos del laboratorio colaborador tienen características que complican las cosas: grabación con cámara de celular, variaciones de iluminación entre sesiones, y ratas de pelaje claro con poco contraste frente al cilindro. El sistema no garantiza robustez ante todas esas condiciones desde el inicio; evaluar qué tan bien funciona con los videos disponibles es parte del trabajo de desarrollo. La hipótesis es que los videos son suficientemente buenos para que el sistema funcione, pero eso se confirma o se ajusta durante las pruebas.

2.2. Trabajos similares

Antes de revisar las herramientas disponibles, vale la pena mirar qué grupos han intentado resolver el mismo problema o uno parecido, y qué resultados reportaron.

2.2.1. DSAAN: clasificación sin umbral en el FST

Meng et al. [11] propusieron DSAAN (Dual-Stream Attention-Augmented Network), un método con dos ramas paralelas que procesan apariencia y movimiento del animal por separado y luego combinan esa información para clasificar el estado cuadro a cuadro. La arquitectura procesa diferencias entre cuadros consecutivos (flujo de movimiento) en paralelo con la imagen original (apariencia), y un módulo de atención pondera la contribución de cada rama según el cuadro.

Con apenas 94 imágenes etiquetadas alcanzó un 88.7 % de precisión en la clasificación. El método fue validado en condiciones controladas; los autores no reportan el comportamiento del sistema con variaciones de iluminación o reflejos del agua, que son condiciones habituales en el laboratorio colaborador.

2.2.2. Red 3D-RCNN para clasificación de tres conductas

Della Valle et al. [2] entrenaron un modelo basado en redes residuales 3D (ResNet-3D) que procesa segmentos de video cortos para capturar el movimiento en el tiempo. A diferencia de la mayoría de los sistemas revisados, este distingue las tres conductas del FST: nado, inmovilidad y escalamiento. Los propios autores señalan que eso lo diferencia de trabajos anteriores, que solo discriminan entre inmovilidad y conductas activas en general.

La red 3D procesa volúmenes espacio-temporales —es decir, secuencias de cuadros como si fueran un volumen tridimensional— en lugar de cuadros individuales. Eso le permite capturar patrones de movimiento que no son visibles en un solo cuadro, como la dirección y velocidad del desplazamiento. La validación fue en condiciones controladas de laboratorio.

2.2.3. DLC como clasificador de conducta en el FST

Sturman et al. [14] mostraron que un clasificador entrenado sobre la representación esquelética producida por DeepLabCut puede eliminar la variabilidad entre evaluadores humanos en el FST. El enfoque consiste en extraer coordenadas de puntos corporales

(nariz, patas, cola) en cada cuadro y usar esas coordenadas como descriptores para un clasificador supervisado. La ventaja es que el clasificador opera sobre una representación compacta del movimiento, no sobre la imagen cruda.

El costo es el esfuerzo de anotación: hay que marcar manualmente los puntos clave en decenas de cuadros antes de entrenar DeepLabCut, lo que implica un trabajo previo considerable.

2.2.4. YOLO-Behaviour: detección y clasificación en dos etapas

Chan et al. [15] presentaron un sistema que separa la detección del animal de la clasificación de su conducta. Primero YOLOv8 detecta y recorta la región del animal en cada cuadro; luego un clasificador independiente opera sobre ese recorte. Separar las dos tareas tiene una ventaja práctica: el clasificador trabaja sobre una imagen más limpia, sin el fondo ni los otros animales como distracción.

Ese enfoque de dos etapas —detección por YOLO seguida de clasificación sobre el recorte— es la arquitectura que adopta este proyecto.

2.3. Herramientas comerciales de análisis conductual por video

2.3.1. EthoVision XT

EthoVision XT (Noldus Information Technology) es el software más usado en laboratorios de neurociencia conductual para seguimiento automático de animales en video [8]. Detecta al animal en cada cuadro por sustracción de fondo y umbralización, y registra posición, velocidad y tiempo de inmovilidad. Tiene módulos para múltiples arenas simultáneas y exporta datos a formatos estadísticos estándar.

Para el FST, EthoVision detecta si el animal se mueve o no, pero no distingue entre nado, inmovilidad y escalamiento [2]. Esa limitación no es menor: sin distinguir las tres conductas, no es posible inferir sobre qué sistema neurotransmisor actúa el fármaco evaluado. Adicionalmente, requiere condiciones de iluminación controladas y su licencia tiene un costo que no es accesible en el contexto académico del IPN.

2.3.2. ANY-maze

ANY-maze (Stoelting Co.) tiene uso similar al de EthoVision. Su interfaz es más accesible y su curva de aprendizaje más corta, pero comparte las mismas limitaciones de fondo: requiere condiciones visuales controladas y no clasifica las subconductas del FST por separado.

2.4. Herramientas de código abierto para análisis del FST

2.4.1. ezTrack

ezTrack [9] es una herramienta de código abierto en Python diseñada para investigadores sin experiencia en programación. Corre sobre Jupyter Notebooks y cuantifica el movimiento comparando la diferencia absoluta entre cuadros consecutivos: los píxeles que cambian más de un umbral se consideran movimiento.

Es útil para experimentos simples donde solo importa si el animal se mueve o no. Hace clasificación binaria (móvil/inmóvil) y asume condiciones de grabación controladas: iluminación uniforme, fondo constante, un solo animal por arena. No fue diseñada para vista cenital con varios cilindros al mismo tiempo, que es exactamente el encuadre que usa el laboratorio.

2.4.2. DBscorer

DBscorer [10] es la herramienta de código abierto más específica para el FST y el test de suspensión de cola (TST). La desarrolló el Indian Institute of Science en MATLAB. Cuantifica la inmovilidad comparando el tamaño de la región del animal entre cuadros consecutivos: si el área varía poco, el animal se considera inmóvil.

Es un paso adelante frente al análisis manual, pero tiene limitaciones que no tienen fácil solución dentro de su arquitectura: es sensible a cambios de iluminación y reflejos del agua, solo distingue móvil/inmóvil sin separar nado de escalamiento, está diseñada para un solo animal a la vez en perspectiva lateral, y no tiene interfaz web. La dependencia de MATLAB además implica una licencia que no siempre está disponible en el contexto académico.

2.5. Técnicas de visión por computadora para seguimiento de animales

2.5.1. Sustracción de fondo

La sustracción de fondo es la técnica más básica para detectar objetos en movimiento en un video con cámara fija. El principio es simple: se estima una imagen de referencia del fondo (el cilindro vacío) y en cada cuadro se calcula la diferencia con esa referencia. Los píxeles con diferencia mayor a un umbral se clasifican como primer plano (la rata).

El modelo de mezcla gaussiana (Gaussian Mixture Model, GMM) [16] es la versión adaptativa más usada: mantiene un modelo estadístico del fondo que se actualiza con cada cuadro, lo que le permite adaptarse a cambios lentos de iluminación. Para el caso de cámara fija que tiene el laboratorio, una alternativa más simple y predecible es calcular la mediana píxel a píxel de los primeros N cuadros. Usar la mediana en vez del promedio evita que una rata inmóvil al inicio del video contamine el modelo de fondo [17].

2.5.2. YOLO (You Only Look Once)

YOLO es un detector de objetos que aplica una sola red neuronal convolucional directamente sobre la imagen completa, produciendo en un solo paso las cajas delimitadoras (*bounding boxes*) y las probabilidades de clase de todos los objetos detectados [18]. A diferencia de detectores en dos etapas como Faster R-CNN, que primero propone regiones candidatas y luego las clasifica, YOLO trata la detección como un problema de regresión unificado sobre una cuadrícula.

La versión YOLOv8 (Ultralytics, 2023) reduce el tiempo de inferencia a 13.5 ms por cuadro en hardware de gama media y supera en precisión a EfficientDet y RetinaNet en benchmarks estándar. Para datasets pequeños, Stadler et al. [19] encontraron que con 100 a 350 imágenes etiquetadas se logra detección casi perfecta en fondos consistentes, lo cual es relevante dado el tamaño de dataset disponible en este proyecto. El modelo no generaliza bien cuando el fondo cambia mucho entre condiciones, lo que hay que considerar durante el entrenamiento.

La separación entre detección y clasificación tiene una ventaja práctica concreta: el clasificador trabaja sobre un recorte limpio del animal, sin el fondo ni los otros cilindros como fuente de error. El cap. 4 detalla por qué ese trade-off es conveniente dado el tamaño del dataset disponible.

2.5.3. DeepLabCut

Sturman et al. [14] mostraron que un clasificador entrenado sobre la representación esquelética producida por DeepLabCut [20] puede eliminar la variabilidad entre evaluadores humanos en el FST. El resultado fue sólido, pero el enfoque requiere marcar manualmente los puntos clave del cuerpo en decenas de cuadros antes de empezar —nariz, patas, cola— lo que implica un trabajo de anotación previo considerable. La pregunta que queda abierta es si esa inversión inicial se justifica frente a enfoques que operan directamente sobre el recorte del animal, sin necesidad de estimar la pose.

2.5.4. Trackers de correlación: CSRT y KCF

Los trackers de correlación aprenden a reconocer cómo se ve el objeto que siguen y en cada cuadro buscan la región más parecida. Son distintos a los detectores: no necesitan saber qué es el objeto, solo cómo se veía en el cuadro anterior. CSRT (*Discriminative Correlation Filter with Channel and Spatial Reliability*) [21] es más preciso pero más lento; KCF (*Kernelized Correlation Filters*) [22] es más rápido pero menos robusto ante cambios bruscos de apariencia.

En el sistema propuesto, los trackers operan como respaldo cuando el detector falla: si YOLO no detecta a la rata en un cuadro, el tracker CSRT mantiene la estimación de posición hasta que el detector vuelve a encontrarla.

2.6. Modelos de clasificación supervisada aplicables al FST

Esta sección revisa los enfoques de machine learning y deep learning que son viables para clasificar las tres conductas del FST (nado, inmovilidad, escalamiento) dado el tamaño de dataset disponible y las condiciones de grabación del laboratorio.

2.6.1. Clasificadores clásicos sobre descriptores de movimiento

Los métodos clásicos de ML (SVM, Random Forest) toman como entrada descriptores numéricos del movimiento —área del contorno, velocidad del centroide, energía cinética, histogramas de flujo óptico— y producen una etiqueta de conducta. Su ventaja es que requieren pocos datos etiquetados y son interpretables. Su limitación es que los descriptores

hay que definirlos a mano, lo que implica conocimiento previo sobre qué características discriminan mejor las conductas.

Para el FST, un descriptor útil es la relación entre el área de movimiento y la posición vertical del centroide: el escalamiento produce movimiento en la mitad superior del cilindro mientras que el nado se distribuye en toda la superficie del agua. Esa diferencia es capturable con descriptores simples [1].

2.6.2. Redes convolucionales con transfer learning

Las CNN [23] son la arquitectura de referencia para clasificación de imágenes. Entrenar una CNN desde cero requiere cientos de miles de imágenes etiquetadas, lo cual no es viable con el dataset disponible. La solución es el *transfer learning* [24]: se toma una red preentrenada en ImageNet (más de un millón de imágenes de 1,000 clases) y se reemplaza solo la última capa de clasificación por una nueva con 3 salidas, una por conducta. Las capas anteriores ya saben detectar bordes, texturas y formas generales, y eso se aprovecha directamente.

ResNet-18 [25] es la arquitectura candidata principal para el clasificador de este proyecto. Tiene 18 capas con conexiones residuales (*skip connections*) que facilitan el entrenamiento en redes profundas al permitir que el gradiente fluya sin desvanecerse. Es ligera —ocupa menos de 50 MB— entrena rápido con datasets pequeños y tiene un buen balance entre precisión y velocidad de inferencia. Si ResNet-18 no alcanza la métrica objetivo ($\kappa > 0{,}80$), se prueba ResNet-50 como alternativa más profunda.

2.6.3. Redes 3D para información temporal

Las redes residuales 3D extienden las convoluciones al dominio temporal: en lugar de filtrar sobre una imagen 2D, filtran sobre un volumen de T cuadros consecutivos. Eso les permite capturar patrones de movimiento que no son visibles en un solo cuadro, como la dirección del nado o la frecuencia de los movimientos de escalamiento. Della Valle et al. [2] usan este enfoque y logran la clasificación de las tres conductas. El costo es computacional: las redes 3D requieren más memoria y tiempo de inferencia que las 2D, lo que puede ser una limitación en hardware sin GPU.

Para este proyecto, la primera opción es ResNet-18 en 2D sobre el recorte del animal (más simple, menor costo computacional). Las redes 3D quedan como alternativa si el clasificador 2D no alcanza la precisión necesaria.

2.7. Criterios de calidad para la evaluación del sistema

La norma ISO/IEC 25010:2023 [12] define los modelos de calidad para sistemas y software, y establece atributos medibles que permiten verificar si un sistema cumple sus objetivos. Para este proyecto, los atributos relevantes son:

- **Functional correctness:** los resultados del sistema deben ser consistentes con las anotaciones del laboratorio. Se mide con el coeficiente kappa de Cohen (κ), que cuantifica la concordancia entre dos evaluadores descontando el azar. El umbral objetivo es $\kappa > 0{,}80$, que corresponde a concordancia casi perfecta según la escala de Landis y Koch.
- **Time behaviour:** el tiempo de análisis de un video de 5 minutos debe ser aceptable para el flujo de trabajo del laboratorio. Se mide en minutos de procesamiento por minuto de video y se define el umbral concreto con el laboratorio durante la fase de pruebas y validación.
- **Usability:** un investigador sin conocimientos de programación debe poder operar el sistema sin ayuda técnica. Se evalúa con una prueba de usabilidad con al menos un usuario real del laboratorio.

Además, el error absoluto medio (MAE) en segundos por animal y conducta completa los criterios de evaluación cuantitativos del clasificador.

2.8. Tabla comparativa

La Tabla 2.1 resume los trabajos y herramientas revisados frente a los criterios más relevantes para el proyecto. La columna *Métricas reportadas* indica qué tan comparable es cada trabajo con los criterios de calidad de la ISO/IEC 25010:2023 [12] adoptados en este proyecto.

Ningún sistema existente cubre los cinco criterios al mismo tiempo. Los comerciales no son abiertos y no clasifican las tres conductas. Los de código abierto no usan aprendizaje profundo ni tienen interfaz web. Los de aprendizaje profundo más recientes sí clasifican las tres conductas pero no tienen plataforma web y no reportan métricas de tiempo de análisis. El sistema propuesto busca cubrir los cinco criterios y documentar explícitamente los atributos de calidad de la ISO/IEC 25010:2023 [12], con la salvedad de que el desempeño real dependerá de la calidad de los videos del laboratorio.

Cuadro 2.1: Comparativa de herramientas y trabajos relacionados con el análisis del FST.

Sistema	**Espec. FST**	**3 cond.**	**DL**	**Web**	**Abierto**	**Métricas reportadas**
EthoVision XT	Parc.	No	No	No	No	Posición, velocidad
ANY-maze	Parc.	No	No	No	No	Posición, velocidad
ezTrack	Parc.	No	No	No	Sí	% movimiento
DBscorer	Sí	No	No	No	Sí	Tiempo inmóvil
DSAAN	Sí	No	Sí	No	No	Precisión 88.7 %
3D-RCNN	Sí	Sí	Sí	No	No	F1 por clase
DeepLabCut+clf	Parc.	Parc.	Sí	No	Sí	κ vs. experto
YOLO-Behaviour	No	—	Sí	No	Sí	Precisión, recall
Propuesta	**Sí**	**Sí**	**Sí**	**Sí**	**Sí**	$\kappa > 0{,}80$, MAE, tiempo

Capítulo 3

Marco teórico

El FST produce videos de ratas nadando en cilindros. Para analizarlos automáticamente hace falta entender qué conductas se observan y qué significan, cómo procesar las imágenes para detectar al animal, qué arquitectura de red neuronal puede clasificar esas conductas con pocos datos etiquetados, y cómo organizar todo eso en un sistema web que el laboratorio pueda usar. Lo que sigue cubre esos cuatro bloques en ese orden.

3.1. El modelo de nado forzado como paradigma preclínico

3.1.1. Depresión y modelos animales

La depresión mayor afecta a más de 280 millones de personas y es una de las principales causas de discapacidad a nivel mundial [3]. Para probar nuevos tratamientos antes de llegar a ensayos clínicos en humanos, los investigadores usan modelos animales que reproducen algunos aspectos del trastorno.

Para que un modelo animal sea válido debe cumplir tres criterios [5]: *validez de constructo* (los mecanismos biológicos involucrados deben parecerse a los del trastorno humano), *validez de cara* (los síntomas observables deben ser similares a los clínicos) y *validez predictiva* (el modelo debe responder a los fármacos que funcionan en pacientes). El FST cumple principalmente la validez predictiva: prácticamente todos los antidepresivos clínicamente aprobados reducen la inmovilidad en este test [6].

3.1.2. La prueba de nado forzado (FST)

La prueba de nado forzado fue propuesta por Porsolt et al. en 1977 [4]. Consiste en poner a la rata en un cilindro con agua del que no puede salir y registrar cómo responde. Al principio intenta escapar; después de un rato empieza a flotar con movimientos mínimos. Esa conducta pasiva —*behavioral despair* o desesperanza conductual— se asocia con la pérdida de motivación propia de la depresión [1].

3.1.3. Conductas que se observan y cómo se interpretan

En el FST se distinguen tres conductas mutuamente excluyentes [1], [6]:

- **Inmovilidad**: el animal flota haciendo solo lo necesario para no hundirse. Es el indicador principal de desesperanza conductual y el parámetro que más se reporta en la literatura.
- **Nado activo**: movimientos de paladas que desplazan al animal horizontalmente. Se asocia al sistema serotoninérgico; los antidepresivos tipo ISRS (como la fluoxetina) aumentan esta conducta específicamente.
- **Escalamiento**: movimientos rápidos hacia las paredes del cilindro, con las patas delanteras tocando o rompiendo la superficie del agua. Se asocia al sistema noradrenérgico; la desipramina lo incrementa. Desde la vista cenital, el escalamiento produce movimiento concentrado en el borde del cilindro, lo que lo distingue visualmente del nado.

Distinguir las tres conductas importa porque cada una refleja la activación de un sistema neurotransmisor distinto, lo que permite inferir el mecanismo de acción del fármaco evaluado. Un sistema que solo detecta si la rata se mueve o no pierde esa información [2].

3.1.4. Protocolo del laboratorio colaborador

En el laboratorio de la ENMyH las ratas se colocan individualmente en cilindros transparentes con agua. Hay dos sesiones: 20 minutos el día 1 (línea base) y 5 minutos el día 2 (después del tratamiento), con 24 horas de diferencia. Los videos se graban desde una vista cenital con cámara de celular, capturando los cuatro cilindros al mismo tiempo. La iluminación y el nivel del agua varían entre experimentos, y con ratas de pelaje claro el

contraste contra el cilindro puede ser bajo. El sistema debe funcionar bajo esas condiciones; los requisitos mínimos de calidad de video se definirán durante las pruebas de validación.

3.2. Procesamiento de video para detección y seguimiento

3.2.1. Representación digital de imágenes y video

Un video se trata como una secuencia de matrices de números. Cada cuadro (*frame*) es una imagen BGR de tres canales (azul, verde, rojo), que es la convención de OpenCV [26]. Un video de 5 minutos a 30 FPS equivale a 9,000 cuadros. Cada píxel tiene un valor entero entre 0 y 255 por canal; la representación completa de un cuadro de resolución $W \times H$ ocupa $W \times H \times 3$ bytes.

3.2.2. Modelo de fondo y sustracción

El primer paso del pipeline es construir un modelo del fondo: una imagen de referencia del cilindro sin la rata. Se calcula como la mediana píxel a píxel de los primeros N cuadros del video:

$$B(x, y) = \text{mediana}\big\{I_1(x, y),\, I_2(x, y),\, \ldots,\, I_N(x, y)\big\} \tag{3.1}$$

Usar la mediana en vez del promedio evita que una rata inmóvil al inicio contamine el modelo. Una vez que se tiene el fondo, en cada cuadro se calcula la diferencia absoluta: los píxeles con diferencia mayor a un umbral son el primer plano (la rata). Este enfoque es más predecible que el modelo de mezcla gaussiana (GMM) [16] para el caso de cámara fija [17].

3.2.3. Umbralización de Otsu

La imagen de diferencias se convierte a binario con el método de Otsu [27]. Este método calcula automáticamente el umbral óptimo T^* que minimiza la varianza intra-clase entre píxeles de fondo y primer plano:

$$T^* = \arg\min_{T} \big[\omega_0(T)\, \sigma_0^2(T) + \omega_1(T)\, \sigma_1^2(T)\big] \tag{3.2}$$

donde ω_0 y ω_1 son las proporciones de píxeles en cada clase y σ_0^2, σ_1^2 sus varianzas. La ventaja de Otsu es que adapta el umbral a cada cuadro sin parámetros fijos, lo que es útil cuando la iluminación varía entre sesiones.

Después de la umbralización se aplica una apertura morfológica (erosión seguida de dilatación) para eliminar ruido pequeño y rellenar huecos en la región detectada.

3.2.4. Detección de contornos e IoU

Sobre la imagen binaria se extraen los contornos con el algoritmo de Suzuki y Abe [28]. De cada contorno se obtiene el área, el rectángulo envolvente (*bounding box*) y el centroide. Para verificar consistencia entre cuadros consecutivos se usa la métrica IoU (*Intersection over Union*):

$$\mathrm{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} \tag{3.3}$$

Un IoU cercano a 1 indica que la detección actual coincide con la del cuadro anterior; si cae por debajo de un umbral se activa el respaldo del tracker de correlación.

3.2.5. Estimación de la línea de agua

La superficie del agua puede generar reflejos que hacen que el animal aparezca duplicado desde la vista superior. Para mitigar ese efecto, el sistema estima la posición vertical de la línea del agua analizando el perfil de intensidad por filas dentro de cada ROI e identificando la fila con el cambio más brusco. Como esa estimación oscila por ruido entre cuadros, se estabiliza con un promedio móvil exponencial (EMA):

$$\hat{y}_t = \alpha \cdot \tilde{y}_t + (1 - \alpha) \cdot \hat{y}_{t-1} \tag{3.4}$$

donde $\tilde{y}_t$ es la estimación del cuadro actual, $\hat{y}_{t-1}$ el valor suavizado anterior y $\alpha \in (0, 1)$ controla la velocidad de respuesta. Un α alto responde rápido a cambios reales pero amplifica el ruido; un α bajo es más estable pero tarda en adaptarse. Esta lógica está encapsulada en la clase `WaterlineSmoother` del módulo `tracker.py`.

3.2.6. Trackers de correlación: CSRT y KCF

Cuando el detector falla en un cuadro —por oclusión parcial, reflejo, o un cuadro donde la rata está demasiado quieta para distinguirse del fondo— el sistema necesita mantener

la estimación de posición sin perder al animal. Para eso usa trackers de correlación como respaldo.

CSRT (*Discriminative Correlation Filter with Channel and Spatial Reliability*) [21] mantiene un mapa de confiabilidad espacial que pondera las regiones más discriminativas del objeto. Es el más preciso de los dos pero también el más lento.

KCF (*Kernelized Correlation Filters*) [22] trabaja eficientemente en el espacio de características circular. Más rápido que CSRT, pero pierde al animal ante cambios bruscos de apariencia: en el FST eso ocurre cuando la rata sale a escalar y vuelve al agua.

El sistema usa CSRT como primera opción y KCF como respaldo si CSRT no está disponible. Si ambos fallan, la última posición conocida se congela con una confianza decreciente.

3.3. Redes neuronales y aprendizaje profundo

3.3.1. Redes neuronales y CNN

Una red neuronal convolucional (CNN) [23] aplica filtros pequeños $\mathbf{K}$ que se deslizan por la imagen detectando patrones locales —bordes, texturas, formas— sin importar su posición en el cuadro. Las capas iniciales aprenden características de bajo nivel; las más profundas las combinan para reconocer formas complejas. Al final, capas completamente conectadas producen las probabilidades de clase.

Los pesos se aprenden durante el entrenamiento con retropropagación y el optimizador Adam [29], que ajusta la tasa de aprendizaje de cada parámetro individualmente usando estimaciones de primer y segundo momento del gradiente.

Entrenar una CNN desde cero requiere cientos de miles de imágenes etiquetadas, lo que no es viable con el dataset disponible. La solución es el *transfer learning*.

3.3.2. Transfer learning

El *transfer learning* [24] consiste en tomar una red ya entrenada en ImageNet (más de un millón de imágenes de 1,000 clases) y adaptar solo su última capa para la tarea nueva. Las capas anteriores ya detectan bordes, texturas y formas generales; ese conocimiento se transfiere directamente al problema del FST. En la práctica se reemplaza la capa de clasificación original por una nueva con 3 salidas —una por conducta— y se re-entrena solo esa parte con los cuadros etiquetados del laboratorio.

Este enfoque reduce los requerimientos de datos cuando el dominio de origen (ImageNet) y el de destino (recortes de rata en FST) comparten características visuales básicas [24].

3.3.3. ResNet y conexiones residuales

ResNet (*Deep Residual Network*) [25] resolvió un problema práctico: las redes de más de 20 capas entrenadas con retropropagación tendían a empeorar porque el gradiente se desvanecía al retroceder por muchas capas consecutivas (*vanishing gradient problem*).

La solución fue agregar conexiones directas que "saltan" una o más capas (*skip connections*). En lugar de que cada bloque aprenda una transformación $\mathcal{H}(\mathbf{x})$ desde cero, aprende solo la diferencia residual $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x} \tag{3.5}$$

Esto permite al gradiente fluir directamente hacia atrás sin atenuarse, lo que hace posible entrenar redes de 50, 101 o 152 capas. Para este proyecto se considera ResNet-18 como primera opción por su balance entre precisión y velocidad de inferencia, y ResNet-50 como alternativa si no se alcanza $\kappa > 0{,}80$.

3.3.4. Clasificación de las tres conductas

El clasificador recibe el recorte de imagen alrededor de la rata (el bounding box), normalizado a 224×224 píxeles. La capa de salida produce tres valores de activación que la función *softmax* convierte en probabilidades:

$$p_c = \frac{e^{z_c}}{\sum_{j=1}^{3} e^{z_j}}, \quad c \in \{\text{nado, inmovilidad, escalamiento}\} \tag{3.6}$$

Se predice la conducta con mayor probabilidad. Durante el entrenamiento, la función de pérdida es la entropía cruzada categórica, que penaliza fuertemente predicciones incorrectas con alta confianza:

$$\mathcal{L} = -\sum_{c=1}^{3} y_c \log p_c \tag{3.7}$$

donde $y_c = 1$ si la etiqueta correcta es la clase c y 0 en caso contrario. Las etiquetas las generan los autores del proyecto usando BORIS, tomando como referencia los registros

conductuales proporcionados por el laboratorio. Esos mismos registros constituyen el gold standard para la validación del sistema.

3.4. Arquitectura del sistema web

3.4.1. Arquitectura cliente-servidor y API REST

El sistema tiene tres capas: el frontend (React.js) corre en el navegador del investigador, el backend (Flask) gestiona la lógica de negocio a través de una API REST, y la base de datos (PostgreSQL) persiste los datos. Esta separación permite que frontend y backend evolucionen de forma independiente [30].

La API REST usa semántica HTTP estándar: `GET` para consultar, `POST` para crear, `PUT`/`PATCH` para actualizar y `DELETE` para eliminar. Cada recurso tiene una URL fija y las respuestas se intercambian en formato JSON.

3.4.2. Procesamiento asíncrono con colas de tareas

Analizar un video de 5 minutos puede tomar varios minutos. Hacerlo dentro del ciclo petición-respuesta HTTP bloquearía al servidor e impediría atender a otros usuarios. La solución es el procesamiento asíncrono: cuando el investigador sube un video, el backend encola la tarea y devuelve inmediatamente una respuesta con el identificador del análisis. Un proceso worker independiente toma la tarea de la cola, ejecuta el pipeline de IA y escribe los resultados en la base de datos. El frontend consulta periódicamente el estado del análisis y actualiza la barra de progreso.

3.4.3. Contenedorización con Docker

Docker permite empaquetar una aplicación con todas sus dependencias en una imagen reproducible [31]. Docker Compose orquesta múltiples contenedores (backend, frontend, base de datos, worker) con un solo comando. La norma ISO/IEC/IEEE 12207:2017 [13] define la portabilidad como un atributo del proceso de despliegue; el uso de Docker satisface ese atributo al garantizar que el sistema se comporta igual en cualquier equipo que tenga Docker instalado.

3.4.4. Persistencia de datos con PostgreSQL

PostgreSQL es un sistema de gestión de bases de datos relacionales de código abierto con soporte para transacciones ACID (*Atomicity, Consistency, Isolation, Durability*), que garantizan que las operaciones de escritura son seguras incluso ante fallos del sistema. El esquema de datos incluye tablas para usuarios, experimentos, sesiones de video y resultados por animal y conducta. Las consultas se abstraen con SQLAlchemy como ORM (*Object-Relational Mapper*), lo que permite cambiar el motor de base de datos sin modificar el código de la aplicación.

3.5. Métricas de evaluación

La evaluación del sistema se basa en métricas que cubren los atributos de calidad definidos en la norma ISO/IEC 25010:2023 [12]: *functional correctness* (precisión de las predicciones), *time behaviour* (tiempo de procesamiento) y *usability* (facilidad de uso para el investigador).

3.5.1. Kappa de Cohen

El coeficiente kappa de Cohen (κ) mide la concordancia entre dos evaluadores —en este caso el sistema y el gold standard del laboratorio— descontando la concordancia que ocurriría por azar:

$$\kappa = \frac{p_o - p_e}{1 - p_e} \tag{3.8}$$

donde p_o es la proporción de acuerdos observados y p_e es la proporción de acuerdos esperados por azar. Un $\kappa = 1$ indica concordancia perfecta; $\kappa = 0$ indica que el acuerdo no supera el azar. Según la escala de Landis y Koch, $\kappa > 0{,}80$ corresponde a concordancia casi perfecta (*almost perfect agreement*), que es el umbral objetivo para este proyecto.

3.5.2. Precisión, recall y F1 por clase

Para cada conducta c se calculan:

$$\text{Precisión}_c = \frac{TP_c}{TP_c + FP_c}, \quad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c} \tag{3.9}$$

$$F1_c = 2 \cdot \frac{\text{Precisión}_c \cdot \text{Recall}_c}{\text{Precisión}_c + \text{Recall}_c} \tag{3.10}$$

donde TP_c, FP_c y FN_c son los verdaderos positivos, falsos positivos y falsos negativos de la clase c respectivamente. El F1 es útil cuando las clases están desbalanceadas, que es esperable en el FST donde la inmovilidad suele dominar en los primeros experimentos.

3.5.3. Error absoluto medio (MAE)

El error absoluto medio mide cuántos segundos se aleja el sistema del tiempo total reportado por el laboratorio para cada conducta:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} \left| \hat{t}_i - t_i \right| \tag{3.11}$$

donde $\hat{t}_i$ es el tiempo predicho por el sistema para el animal i y t_i es el tiempo registrado manualmente. El MAE se reporta en segundos por animal para que el laboratorio pueda interpretar el error en términos de su flujo de trabajo real.

3.5.4. Tiempo de análisis

El tiempo de procesamiento de un video de 5 minutos se mide en minutos de cómputo y se reporta como relación minuto de video / minuto de procesamiento. El umbral aceptable se define con el laboratorio durante el Sprint 6, conforme al atributo de *time behaviour* de la ISO/IEC 25010:2023 [12].

Capítulo 4

Análisis

En este capítulo se definen los requerimientos del sistema a partir de los prototipos de interfaz desarrollados para el flujo de trabajo del laboratorio colaborador y de reuniones con el Dr. César Augusto Sandino Reyes López, investigador del Laboratorio de Neurociencia Conductual de la ENMyH-IPN. Se documentan también las reglas de negocio, los riesgos identificados y la factibilidad del proyecto desde cuatro ángulos: tecnológico, operativo, económico y legal.

Los requerimientos funcionales se organizan por módulo de la interfaz. Los tiempos de respuesta y criterios de rendimiento se derivan de los atributos de calidad definidos en la norma ISO/IEC 25010:2023 [12]: *time behaviour*, *functional correctness* y *usability*. El ciclo de levantamiento sigue los procesos de definición de requerimientos de la norma ISO/IEC/IEEE 12207:2017 [13].

4.1. Requerimientos

4.1.1. Requerimientos Funcionales (RF)

Los requerimientos funcionales están organizados por módulo de la interfaz con tres niveles de prioridad: **Alta** (sin esto el prototipo no funciona), **Media** (importante pero puede quedar para una iteración posterior) y **Baja** (deseable pero no urgente). Cada requerimiento se derivó directamente de los prototipos de interfaz diseñados [32], [33].

Cuadro 4.1: Requerimientos Funcionales (RF).

ID	Módulo	Descripción	Prior.	Actor
RF-01	Autenticación	El sistema debe permitir iniciar sesión con correo electrónico institucional (`@ipn.mx`) y contraseña. Las credenciales incorrectas muestran un mensaje de error sin revelar cuál campo falló.	Alta	Investigador / Admin
RF-02	Autenticación	El sistema debe cerrar la sesión del usuario de forma segura, invalidando el token JWT activo. La opción de cierre de sesión debe ser accesible desde el menú de usuario en todas las pantallas.	Alta	Investigador / Admin
RF-03	Autenticación	El sistema debe ofrecer recuperación de contraseña enviando un enlace de restablecimiento al correo registrado.	Media	Investigador
RF-04	Autenticación	El sistema debe gestionar dos roles con permisos distintos: Investigador (acceso a sus propios experimentos) y Administrador (acceso a todo el sistema).	Alta	Sistema
RF-05	Gestión de usuarios	El Administrador debe poder crear, modificar y desactivar cuentas de usuario desde el panel de administración. La creación requiere nombre, correo y contraseña temporal; el usuario la cambia al primer ingreso.	Alta	Admin
RF-06	Gestión de usuarios	El Investigador debe poder actualizar su nombre, correo y contraseña desde su perfil de usuario.	Media	Investigador

ID	Módulo	Descripción	Prior.	Actor
RF-07	Gestión de usuarios	**[PENDIENTE — entrevista con el Dr. Sandino]:** ¿Los investigadores pueden registrarse solos con confirmación por correo, o solo el Administrador puede crear cuentas? Definir antes de iniciar la implementación del módulo de autenticación.	Alta	Admin
RF-08	Carga de video	El sistema solo acepta archivos en formato `.mp4`. Cualquier otro formato se rechaza con un mensaje de error específico antes de intentar subirlo al servidor.	Alta	Sistema
RF-09	Carga de video	El sistema debe permitir subir uno o dos videos por experimento: Día 1 (sesión basal, 20 min) y Día 2 (sesión post-tratamiento, 5 min). Ambos son opcionales individualmente; el análisis usa los que se proporcionen.	Alta	Investigador
RF-10	Carga de video	El sistema debe validar que el archivo subido sea un video reproducible antes de encolarlo. Si la validación falla, lo rechaza con código de error y mensaje claro.	Alta	Sistema
RF-11	Carga de video	Al crear un experimento el investigador debe registrar: nombre del experimento, fecha, número esperado de animales (típicamente 4), tratamiento o condición experimental, y notas adicionales opcionales.	Alta	Investigador
RF-12	Carga de video	El análisis debe iniciarse automáticamente al completarse la carga y validación del video, sin intervención del investigador.	Alta	Sistema

ID	Módulo	Descripción	Prior.	Actor
RF-13	Análisis conductual	El pipeline de análisis ejecuta cuatro etapas secuenciales: preprocesamiento del video (decodificación y corrección de contraste), detección de cilindros, seguimiento de animales (*tracking*) y clasificación de conductas. Cada etapa tiene un indicador de estado en la pantalla de progreso.	Alta	Sistema
RF-14	Análisis conductual	El sistema debe detectar automáticamente la posición de los cuatro cilindros en el encuadre con una confianza mínima de 0.70. Si no se alcanza ese umbral, el pipeline se detiene y genera un reporte de diagnóstico.	Alta	Sistema
RF-15	Análisis conductual	Mientras el análisis corre, el investigador debe ver el progreso en tiempo real con una barra de porcentaje y la etapa activa. El análisis continúa en segundo plano aunque el investigador cierre la pestaña.	Alta	Investigador
RF-16	Análisis conductual	Si el análisis falla, el sistema muestra el código de error, la descripción del problema y un botón para descargar el reporte de diagnóstico en PDF. El investigador no puede reiniciar el análisis desde esta pantalla (ver RN-04).	Alta	Investigador
RF-17	Análisis conductual	El sistema debe clasificar cuadro a cuadro la conducta de cada animal: nado activo, inmovilidad o escalamiento [1]. Las conductas son mutuamente excluyentes (ver RN-07).	Alta	Sistema

ID	Módulo	Descripción	Prior.	Actor
RF-18	Resultados	La pantalla de resultados debe mostrar el tiempo total en segundos y el porcentaje de cada conducta por animal y por sesión, en una tabla descargable.	Alta	Investigador
RF-19	Resultados	El sistema debe ofrecer un desglose de conductas por minuto para cada animal.	Media	Investigador
RF-20	Resultados	Cuando ambos videos del mismo experimento estén procesados, el sistema debe mostrar una comparación Día 1 vs. Día 2 con barras de distribución por animal.	Alta	Investigador
RF-21	Resultados	El investigador debe poder descargar el reporte de resultados en PDF y en CSV desde la pantalla de resultados y desde el dashboard.	Alta	Investigador
RF-22	Dashboard	El dashboard debe mostrar la lista de experimentos del investigador activo con nombre, fecha, tratamiento, sesión, estado (en proceso, completado, error) y accesos directos a resultados y descarga.	Alta	Investigador

ID	Módulo	Descripción	Prior.	Actor
RF-23	Dashboard	El sistema debe mostrar un aviso visible cuando un video está a menos de 7 días de ser eliminado automáticamente, indicando el nombre del experimento y la fecha de vencimiento exacta. El período de 7 días da tiempo suficiente para descargar reportes antes del borrado, conforme al atributo *user error protection* de la ISO/IEC 25010:2023 [12], que establece que el sistema debe proteger al usuario de consecuencias no intencionadas de sus acciones u omisiones.	Media	Sistema
RF-24	Dashboard	El sistema debe borrar automáticamente los archivos de video 30 días después de que el análisis termine exitosamente, notificando al investigador con un aviso en la interfaz antes de que ocurra (ver RN-05).	Media	Sistema
RF-25	Dashboard	Los resultados y reportes deben conservarse indefinidamente, aunque el video original ya haya sido borrado.	Alta	Sistema
RF-26	Dashboard	El investigador debe poder eliminar un experimento desde el dashboard. La acción es permanente e irreversible y requiere confirmación explícita (ver RN-09).	Media	Investigador
RF-27	Administración	El Administrador debe poder ver la lista de todos los usuarios con correo, rol, estado (activo/inactivo), último acceso y número de experimentos. La lista es filtrable por rol y estado.	Media	Admin

ID	Módulo	Descripción	Prior.	Actor
RF-28	Administración	El Administrador debe poder monitorear el estado del sistema: porcentaje de uso del disco, espacio disponible en GB, tamaño de la cola de procesamiento, número de análisis activos y ETA estimado. El sistema alerta cuando el disco supera el 80 % de uso.	Baja	Admin
RF-29	Administración	El Administrador puede ver y gestionar los experimentos de cualquier investigador (ver RN-08).	Media	Admin

4.1.2. Requerimientos No Funcionales (RNF)

Los tiempos de respuesta se derivan del atributo *time behaviour* de la norma ISO/IEC 25010:2023 [12], que establece que un sistema debe responder dentro de los tiempos que el usuario percibe como aceptables para cada tipo de operación. Para operaciones interactivas cortas (navegación, filtros, consultas) el umbral estándar es de 1 a 3 segundos; para operaciones de procesamiento intensivo (análisis de video) el tiempo aceptable se determina con el usuario durante las pruebas [12].

Cuadro 4.2: Requerimientos No Funcionales (RNF).

ID	Categoría	Descripción	Métrica / Criterio
RNF-01	Rendimiento	Las operaciones de navegación e interacción con la interfaz (cargar el dashboard, filtrar experimentos, abrir resultados) deben responder en menos de 3 s. Derivado del atributo *time behaviour* de la ISO/IEC 25010:2023 [12].	Tiempo de respuesta $\leq$ 3 s en el 95 % de las solicitudes en condiciones normales de red.

ID	Categoría	Descripción	Métrica / Criterio
RNF-02	Rendimiento	**[PENDIENTE — entrevista con el Dr. Sandino]:** El tiempo máximo aceptable para analizar un video de 5 min (Día 2, resolución típica de celular) se definirá con el laboratorio durante la fase de pruebas y validación.	Tiempo máximo a medir y documentar durante las pruebas con videos reales del laboratorio.
RNF-03	Disponibilidad	El sistema debe estar disponible en el horario de trabajo del laboratorio. No se requiere disponibilidad 24/7 para el prototipo.	95 % de disponibilidad de lunes a viernes, 8:00–20:00 h.
RNF-04	Seguridad	Solo los usuarios autenticados pueden acceder al sistema. La autenticación usa tokens JWT con tiempo de expiración configurable. Los datos de cada investigador no son visibles para otros usuarios sin autorización explícita.	Autenticación con JWT. Datos aislados por usuario en la base de datos. Verificado en las pruebas de integración de la fase de implementación.
RNF-05	Seguridad	Las contraseñas deben almacenarse cifradas con sal, nunca en texto plano. Las contraseñas temporales asignadas por el Admin se marcan para cambio obligatorio al primer ingreso.	Hash con sal usando bcrypt. Política de contraseña temporal verificada en prueba funcional.
RNF-06	Usabilidad	Un investigador sin experiencia en programación debe poder crear un experimento, subir un video, esperar el análisis y descargar el reporte sin necesitar ayuda técnica. Derivado del atributo *usability* de la ISO/IEC 25010:2023 [12].	Prueba con al menos un usuario real del laboratorio durante la fase de validación y robustez. Tasa de completación de tarea $\geq 90\,\%$ sin asistencia.

ID	Categoría	Descripción	Métrica / Criterio
RNF-07	Compatibilidad	El sistema debe funcionar desde Chrome, Firefox y Edge en sus versiones actuales, sin instalar nada en el equipo del usuario.	Pruebas en los tres navegadores en sus versiones actuales al cierre de la fase de implementación de la interfaz web.
RNF-08	Almacenamiento	Los videos se borran automáticamente 30 días después de que el análisis termina exitosamente. Los resultados y reportes se conservan indefinidamente. El sistema alerta al usuario cuando un video está a menos de 7 días de ser borrado.	Tarea programada (*cron job*) diaria. Aviso visible en dashboard con indicador de días restantes.
RNF-09	Almacenamiento	El Administrador recibe una alerta en el panel cuando el uso del disco supera el 80 %. A los 90 %, el sistema elimina automáticamente los videos más antiguos, notificando al investigador correspondiente.	Umbral de alerta: 80 %. Umbral de limpieza automática: 90 %. Los valores 80/90 % son umbrales estándar de gestión de capacidad en sistemas de almacenamiento [32]; dejan margen operativo antes de que el disco se llene y evitan pérdida de datos por falta de espacio.
RNF-10	Portabilidad	El sistema debe poder levantarse con Docker Compose en cualquier equipo con Docker instalado, sin dependencias adicionales en el sistema anfitrión. Derivado del proceso de despliegue de la ISO/IEC/IEEE 12207:2017 [13].	Despliegue documentado y verificado en al menos dos entornos distintos.

ID	Categoría	Descripción	Métrica / Criterio
RNF-11	Escalabilidad	El módulo de clasificación debe poder reconfigurarse para soportar paradigmas conductuales distintos al FST sin modificar la plataforma web.	Diseño modular verificado en la revisión de arquitectura al cierre del bloque de diseño.
RNF-12	Mantenibilidad	El código debe estar documentado y separado en módulos independientes (backend, pipeline de IA, frontend), siguiendo PEP 8 en Python y ESLint en JavaScript.	Revisión de calidad de código en cada cierre de bloque de implementación.
RNF-13	Concurrencia	**[PENDIENTE — entrevista con el Dr. Sandino]:** Número de usuarios y análisis simultáneos que debe soportar el sistema, e infraestructura disponible en el laboratorio. Definir antes de la fase de pruebas de carga.	Pendiente. Se documenta antes de iniciar las pruebas de carga.

4.1.3. Reglas de negocio

Las reglas de negocio son los límites que el sistema siempre debe respetar, independientemente de cómo esté implementado. Algunas vienen del protocolo experimental del FST, otras del diseño de la interfaz y otras del contexto institucional del laboratorio [32]. Los requerimientos que dependen de estas reglas las referencian explícitamente.

Cuadro 4.3: Reglas de negocio.

ID	Regla	Justificación / RF relacionados
RN-01	Las credenciales incorrectas muestran un mensaje de error genérico que no especifica si el correo o la contraseña es lo que falló.	Seguridad básica contra enumeración de usuarios. Ver RF-01.

ID	Regla	Justificación / RF relacionados
RN-02	Un video pertenece a exactamente un experimento. No se puede reutilizar un video ya subido en un experimento distinto.	Trazabilidad de resultados. Ver RF-09.
RN-03	Un experimento tiene como máximo dos videos: uno del Día 1 (20 min) y uno del Día 2 (5 min).	Así funciona el protocolo FST del laboratorio [5]. Ver RF-09.
RN-04	El análisis lo inicia el sistema automáticamente al terminar la carga. El investigador no puede iniciarlo, pausarlo, cancelarlo ni reiniciarlo desde la interfaz.	Evita estados inconsistentes en la cola. Ver RF-12, RF-16.
RN-05	Los videos se borran automáticamente 30 días después de que el análisis termina exitosamente. La acción es irreversible. El sistema avisa con al menos 7 días de anticipación.	Control de almacenamiento. Ver RF-24, RF-23, RNF-08.
RN-06	Los resultados y reportes (PDF y CSV) se conservan indefinidamente aunque el video original ya haya sido borrado.	Los datos de investigación tienen valor a largo plazo. Ver RF-25.
RN-07	En cada cuadro del video, a cada animal se le asigna exactamente una conducta: nado activo, inmovilidad o escalamiento. No pueden coexistir dos conductas al mismo tiempo para el mismo animal.	El FST define estas conductas como mutuamente excluyentes [1], [6]. Ver RF-17, RF-18.
RN-08	Cada investigador solo puede ver sus propios experimentos y resultados. El Administrador puede ver y gestionar los de cualquier investigador.	Control de acceso. Ver RF-04, RF-29.
RN-09	Borrar un experimento es permanente e irreversible: se eliminan el video, los resultados y los reportes. El sistema requiere confirmación explícita antes de ejecutarlo.	Previene borrados accidentales. Ver RF-26.

ID	Regla	Justificación / RF relacionados
RN-10	El sistema rechaza cualquier archivo que no sea `.mp4` antes de intentar subirlo al servidor. La validación de formato ocurre en el cliente.	Evita consumir recursos con archivos incompatibles. Ver RF-08, RF-10.
RN-11	La detección de cilindros requiere una confianza mínima de 0.70. Si no se alcanza ese umbral, el pipeline se detiene y genera un reporte de diagnóstico descargable automáticamente. El umbral de 0.70 es un valor conservador estándar en detección con YOLOv8 para fondos consistentes [19]; se ajustará con base en las pruebas con los videos reales del laboratorio durante la fase de validación.	Evita que el sistema produzca resultados incorrectos sin advertir al usuario. Ver RF-14, RF-16.
RN-12	La comparación Día 1 vs. Día 2 solo se genera si ambos videos del mismo experimento fueron procesados exitosamente.	No tiene sentido comparar con datos incompletos. Ver RF-20.

4.2. Análisis de las herramientas a utilizar

La elección del *stack* tecnológico se hizo evaluando madurez, compatibilidad, disponibilidad de documentación y experiencia previa del equipo. Las herramientas se agrupan en cuatro áreas.

4.2.1. Detección de objetos y seguimiento

Cuadro 4.4: Técnicas de detección de objetos y seguimiento.

Herramienta	**Características principales**	**Por qué se eligió**
OpenCV 4.x [26]	Biblioteca estándar para procesamiento de imágenes y video en Python. Incluye sustracción de fondo, umbralización de Otsu, morfología, contornos y los trackers CSRT y KCF.	Es la herramienta de referencia para este tipo de trabajo y es compatible con YOLOv8 y TensorFlow sin configuración extra.
Ultralytics YOLOv8 [18]	Detector de objetos estado del arte. Con 100–350 imágenes etiquetadas logra detección casi perfecta en fondos consistentes [19].	El detector de rata ya fue entrenado (`weights/rat.pt`) y su viabilidad técnica está confirmada con los videos del laboratorio.
Sustracción de fondo por mediana [17]	El fondo se estima como la mediana píxel a píxel de los primeros N cuadros, evitando que una rata inmóvil al inicio contamine el modelo.	Más simple y predecible que el GMM [16] para el caso de cámara fija. No requiere parámetros de color ni modelos adaptativos, lo que reduce la sensibilidad a variaciones de iluminación entre sesiones.
Tracker CSRT [21]	Tracker de correlación con mapa de confiabilidad espacial. Más preciso que KCF aunque más lento.	Primera opción de respaldo cuando YOLO no detecta a la rata en un cuadro. KCF [22] como respaldo secundario.

4.2.2. Aprendizaje profundo para clasificación de conductas

Cuadro 4.5: Arquitecturas de aprendizaje profundo para clasificación de conductas.

Herramienta / Arquitectura	**Características principales**	**Por qué se eligió**
ResNet-18 [25]	Red con 18 capas y conexiones residuales, preentrenada en ImageNet. Ligera (< 50 MB) y rápida de entrenar con datasets pequeños via transfer learning [24].	Primera opción para el clasificador de conducta. Su viabilidad se confirmará durante la fase de implementación del clasificador.
ResNet-50 [25]	Versión más profunda (50 capas). Mayor capacidad que ResNet-18 a costa de más tiempo de entrenamiento e inferencia.	Se usará si ResNet-18 no alcanza $\kappa > 0{,}80$.
TensorFlow 2.x / Keras [34]	Framework de aprendizaje profundo con API de alto nivel para fine-tuning de modelos preentrenados.	Ya se usa en el proyecto y es compatible con Docker para el despliegue.
scikit-learn [35]	Biblioteca de ML para métricas y validación cruzada.	Se usa para calcular el coeficiente κ de Cohen, F1 por clase y MAE.
BORIS [36]	Software de anotación conductual cuadro a cuadro. Exporta en formatos compatibles con Python.	Los autores del proyecto anotan los cuadros de video con BORIS usando los registros conductuales del laboratorio como referencia, generando el dataset de entrenamiento y el gold standard de validación.

4.2.3. Desarrollo web e infraestructura

Cuadro 4.6: Herramientas de desarrollo web e infraestructura.

Herramienta	**Características principales**	**Por qué se eligió**
Flask (Python) [30]	Microframework ligero para APIs REST. Extensiones para JWT (Flask-JWT-Extended), ORM (SQLAlchemy) y tareas asíncronas.	Suficiente para el tamaño del prototipo. Integra el pipeline de IA sin necesitar un framework más pesado.
React.js + Vite	Framework frontend basado en componentes. Vite reduce los tiempos de compilación durante el desarrollo.	Adecuado para la barra de progreso en tiempo real y la interfaz reactiva que requiere el sistema.
PostgreSQL	Base de datos relacional con transacciones ACID y código abierto.	Cubre los patrones de acceso del sistema y se integra con SQLAlchemy.
Docker + Docker Compose [31]	Empaqueta todos los servicios (backend, frontend, base de datos, worker) en contenedores y los levanta con un solo comando.	Cumple el atributo de portabilidad de la ISO/IEC/IEEE 12207:2017 [13]. El laboratorio no instala nada más allá de Docker.

4.3. Análisis de riesgos

La identificación de riesgos sigue el modelo de la norma ISO 31000 **iso31000**, que define el riesgo como la combinación entre la probabilidad de un evento y su impacto. Para cada riesgo se documenta una acción concreta de mitigación.

Cuadro 4.7: Matriz de riesgos del proyecto.

ID	Descripción	Prob.	Impacto	Nivel	Mitigación
R-01	Videos con reflejos, iluminación variable o ratas de pelaje claro que hacen fallar el pipeline (confianza de detección $<$ 0,70, ver RN-11).	Alta	Alto	Alto	Definir requisitos mínimos de calidad de video con el laboratorio. El sistema aplica corrección CLAHE (*Contrast Limited Adaptive Histogram Equalization*) [26] como primer intento automático de mejora de contraste antes de abortar el pipeline; si después de CLAHE la confianza sigue por debajo de 0.70, se genera el reporte de diagnóstico (ver RN-11).
R-02	El dataset de cuadros etiquetados no alcanza para entrenar un clasificador con $\kappa > 0{,}80$.	Alta	Alto	Alto	Ampliar el dataset anotando más cuadros con BORIS. Usar data augmentation. Si escalamiento no se distingue con precisión, reportarlo junto con nado como "conducta activa" (ver alcance, cap. 1).

ID	Descripción	Prob.	Impacto	Nivel	Mitigación
R-03	El laboratorio no proporciona los registros conductuales en el tiempo necesario para el entrenamiento y validación del clasificador.	Media	Alto	Medio	Acordar un calendario de entrega de registros con el laboratorio al inicio de la fase de implementación y preparar los materiales de anotación con anticipación para aprovechar cada sesión con el Dr. Sandino.
R-04	El tiempo de análisis de un video de 5 min supera lo que el laboratorio puede asumir en su flujo de trabajo.	Media	Medio	Medio	Medir tiempos por etapa del pipeline. Evaluar reducción de resolución o uso de GPU. El umbral aceptable se define con el laboratorio durante la fase de pruebas (ver RNF-02).
R-05	La integración del módulo de IA con el backend via cola asíncrona resulta más compleja de lo esperado.	Media	Medio	Medio	Definir contratos de API al inicio de la fase de implementación y hacer una prueba de integración de extremo a extremo antes de agregar más funcionalidad.
R-06	El disco del servidor se llena por acumulación de videos durante los 30 días de retención.	Baja	Medio	Bajo	El sistema incluye alertas al 80 % de uso de disco y limpieza automática al 90 % (ver RF-28, RNF-09).

ID	Descripción	Prob.	Impacto	Nivel	Mitigación
R-07	El laboratorio solicita cambios en los requerimientos a mitad del desarrollo.	Baja	Alto	Medio	Diseño modular del clasificador para agregar clases. Gestionar cambios dentro del proceso de revisión de sprint.
R-08	El escalamiento y el nado activo se ven tan parecidos desde la vista cenital que el modelo no los distingue con suficiente precisión.	Alta	Medio	Medio	Contemplado en el alcance del cap. 1: ambas conductas se reportan juntas como "conducta activa" si no se alcanza la precisión objetivo.

4.4. Análisis de factibilidad

4.4.1. Factibilidad tecnológica

La factibilidad tecnológica consiste en evaluar si el proyecto puede desarrollarse con la infraestructura técnica, el conocimiento especializado y los recursos disponibles. El objetivo es determinar si las tecnologías actuales y el personal son suficientes para implementar el prototipo de forma eficiente y sostenible [32].

Evaluación de recursos actuales y brechas tecnológicas

El equipo de desarrollo cuenta con experiencia en las herramientas del stack tecnológico seleccionado —Python, OpenCV, TensorFlow, Flask y React— adquirida en cursos de la carrera en la ESCOM. Todas las herramientas son de código abierto, están bien documentadas y tienen comunidades activas que reducen el riesgo de incompatibilidades.

La brecha tecnológica más relevante identificada es el dataset de cuadros etiquetados para entrenar el clasificador de conducta. Con el número de videos disponibles actualmente en el laboratorio, ese dataset requiere un esfuerzo de anotación manual con BORIS que es viable pero no trivial. La estrategia de transfer learning con ResNet-18 reduce ese requerimiento a unos pocos cientos de cuadros por clase, lo que está dentro del alcance

del proyecto [11], [24]. Para el entrenamiento del modelo, si el laboratorio no dispone de GPU, Google Colab ofrece hasta 12 horas de GPU por sesión de forma gratuita, lo que es suficiente para el fine-tuning de ResNet-18.

En cuanto al hardware para el despliegue, el sistema no requiere infraestructura especializada: cualquier equipo con Docker instalado puede correr el prototipo, lo que elimina dependencias de hardware específico en el laboratorio.

Simulación, modelado y pruebas

Antes de la implementación completa del sistema, se realizaron pruebas preliminares de detección de objetos con YOLOv8 usando un dataset inicial de los videos del laboratorio. Los resultados de esa prueba confirman que el enfoque es viable con las condiciones de grabación reales del laboratorio: vista cenital, cuatro cilindros simultáneos, cámara de celular [19]. Esa prueba reduce el riesgo técnico de la etapa de detección antes de comprometer recursos en el desarrollo completo.

Para el clasificador de conducta, la literatura reporta precisiones superiores al 88 % con datasets de menos de 100 imágenes cuando se usa transfer learning desde ImageNet [11], lo que da respaldo empírico a la viabilidad de esa etapa con el dataset disponible.

Evaluación de compatibilidad e integración

El prototipo tiene una arquitectura modular que separa el pipeline de IA (Python, OpenCV, TensorFlow) de la plataforma web (Flask, React, PostgreSQL) mediante una cola de tareas asíncrona. Esa separación permite desarrollar y probar cada módulo de forma independiente y facilita la integración progresiva conforme avanza el desarrollo.

La contenedorización con Docker garantiza que el sistema se comporta igual en el equipo de desarrollo, en el laboratorio y en cualquier servidor institucional, lo que elimina la clase más común de problemas de compatibilidad en el despliegue [31].

4.4.2. Factibilidad operativa

El objetivo de la factibilidad operativa es establecer si el sistema propuesto puede emplearse adecuadamente en el contexto académico y experimental para el que fue concebido. Este proyecto responde a una necesidad concreta expresada por el Dr. César Augusto Sandino Reyes López, investigador del Laboratorio de Neurociencia Conductual de la Escuela

Nacional de Medicina y Homeopatía (ENMyH) del IPN, quien realiza experimentos de nado forzado de forma regular y actualmente analiza los videos de forma manual.

Para respaldar esta factibilidad, se realizan reuniones con el Dr. Sandino en las que se aplica un instrumento de recolección de datos —una entrevista estructurada con preguntas abiertas— orientado a obtener información concreta sobre la demanda del sistema, los indicadores más relevantes para el laboratorio, las condiciones reales de operación y los criterios de aceptación desde la perspectiva del usuario experto. Los resultados de esa entrevista se incorporan a este documento antes de la entrega final; el instrumento completo se incluye en los anexos.

[PENDIENTE — entrevista con el Dr. Sandino]:

Con base en las reuniones de arranque del proyecto y en el análisis del protocolo experimental del laboratorio (cap. 1), se pueden anticipar las siguientes ventajas operativas del sistema: reducción del tiempo de análisis de varias horas a minutos, estandarización del criterio de evaluación entre sesiones y evaluadores, generación automática de reportes en los formatos que ya usa el laboratorio (PDF, CSV) y posibilidad de comparar grupos y tratamientos desde una interfaz centralizada.

4.4.3. Factibilidad económica

El proyecto no requiere licencias de software. Todas las herramientas del stack son de código abierto con licencias compatibles con uso académico. El desarrollo lo realiza el equipo del TT sin costo adicional, y la infraestructura de despliegue usa el equipo del laboratorio o las computadoras del equipo de desarrollo.

Cuadro 4.8: Análisis de costos del proyecto.

Concepto	**Costo estimado**
Licencias de software (Python, Flask, React, PostgreSQL, Docker, YOLOv8, OpenCV, BORIS)	$0.00 — todas de código abierto con licencias compatibles con uso académico.
Entrenamiento del modelo (GPU)	$0.00 — Google Colab gratuito (hasta 12 h de GPU por sesión).
Infraestructura de despliegue	Sin costo adicional — equipo del laboratorio o computadora del equipo de TT.

Concepto	**Costo estimado**
Almacenamiento de videos	≈500 MB por experimento de 5 min en MP4. La política de eliminación automática a 30 días (RN-05) evita que el espacio crezca sin control.
Costo total de herramientas y desarrollo	**$0.00**

El proyecto no requiere financiamiento externo ni aprobaciones institucionales para iniciar el desarrollo.

4.4.4. Factibilidad legal

Todas las herramientas del proyecto usan licencias de código abierto que permiten el uso académico sin restricciones: PSF (Python), BSD (Flask), MIT (React), PostgreSQL license (PostgreSQL), Apache 2.0 (Docker, OpenCV), AGPL-3.0 para uso no comercial (YOLOv8) y GNU GPL v3 (BORIS [36]).

Los videos son propiedad del grupo de investigación de la ENMyH-IPN y su uso está respaldado por la colaboración formal entre los directores del TT y el laboratorio. El equipo de desarrollo no interviene en ningún experimento con animales; eso lo hace el laboratorio bajo la NOM-062-ZOO-1999 [7]. El desarrollo del sistema cumple con los Lineamientos de Propiedad Intelectual y Transferencia de Conocimiento del IPN (2023).

4.5. Análisis de sostenibilidad

4.5.1. Sostenibilidad ambiental

Reducir el análisis manual quita la necesidad de que los investigadores vayan al laboratorio solo para cronometrar videos. Al ser web, el sistema se puede usar desde cualquier lugar. La política de borrar videos a los 30 días evita que el disco crezca sin control. Según las directrices de la norma ISO 14064-1 **iso14064**, el consumo energético de procesar un video de 5 min en una computadora local es marginal comparado con las horas de trabajo que reemplaza.

4.5.2. Sostenibilidad social e institucional

El sistema está hecho para el laboratorio de la ENMyH, pero su diseño modular permite adaptarlo a otros paradigmas conductuales (laberinto en cruz elevada, campo abierto, etc.) reconfigurando solo el módulo de clasificación. Eso abre la puerta a que otros grupos del IPN lo usen sin desarrollarlo desde cero.

Si funciona bien, el Dr. Sandino deja de cronometrar videos a mano. Eso es lo que el laboratorio necesita; cualquier beneficio más amplio —reproducibilidad entre grupos, escalabilidad a otros paradigmas— depende de que primero funcione para el caso para el que fue construido.

4.5.3. Mantenibilidad del sistema

El sistema se entrega como prototipo funcional. Para mantenerlo hace falta conocimiento básico de Python y Docker. La documentación técnica de este documento y el `README` del repositorio están diseñados para que cualquiera que retome el proyecto pueda entenderlo sin el equipo original. Las herramientas principales (YOLOv8, Flask, React, PostgreSQL) tienen desarrollo activo y comunidades grandes.

Capítulo 5

Diseño

Este capítulo describe las decisiones de diseño del sistema: cómo están organizados sus módulos, cómo se estructura la base de datos, cómo se comunican los componentes a través de la API REST, y cómo se ve la interfaz desde la perspectiva del investigador. También incluye el diseño del pipeline de análisis conductual: el flujo de procesamiento de video, la estrategia de separación por sujeto y la arquitectura del clasificador.

La decisión de diseño más importante fue separar el worker del backend en un proceso independiente. La opción más simple habría sido ejecutar el pipeline dentro del mismo servidor Flask, pero eso significa bloquear todas las peticiones HTTP mientras un video de 20 minutos se procesa. El worker resuelve ese problema a costa de mayor complejidad en el despliegue: hace falta una cola de tareas, un proceso adicional y comunicación entre procesos. Esa complejidad extra está justificada porque el laboratorio puede tener varios análisis en cola al mismo tiempo.

5.1. Arquitectura general del sistema

El sistema tiene una arquitectura cliente-servidor de tres capas separadas en contenedores Docker independientes: frontend, backend y base de datos. Un cuarto contenedor —el worker— ejecuta el pipeline de análisis de forma asíncrona, sin bloquear al servidor web mientras procesa los videos.

Figura 5.1: Arquitectura general del sistema [autoría propia].

La Figura 5.1 muestra la arquitectura. Las responsabilidades de cada componente son:

- **Frontend (React.js + Vite):** corre en el navegador del investigador. Se comunica con el backend exclusivamente mediante la API REST usando JSON. No tiene acceso directo a la base de datos ni al sistema de archivos del servidor.
- **Backend (Flask):** expone la API REST, gestiona la autenticación con JWT, valida los archivos subidos, encola las tareas de análisis y devuelve los resultados al frontend. Usa SQLAlchemy como ORM para interactuar con PostgreSQL.
- **Worker (Python):** proceso independiente que consume tareas de la cola, ejecuta el pipeline completo de análisis (preprocesamiento, detección, seguimiento y clasificación) y escribe los resultados en la base de datos. Al correr en un proceso separado, el servidor web sigue respondiendo a otras peticiones mientras el video se procesa.
- **Base de datos (PostgreSQL):** almacena usuarios, experimentos, sesiones de video, resultados por animal y conducta, y reportes generados. Los archivos de video se guardan en el sistema de archivos del servidor, no en la base de datos.

La separación en contenedores cumple con el atributo de portabilidad de la ISO/IEC/IEEE 12207:2017 [13]: el sistema puede desplegarse en cualquier equipo con Docker instalado sin dependencias adicionales. La separación entre worker y backend también satisface el RF-12 (análisis automático en segundo plano) y permite escalar el número de workers de forma independiente si el volumen de análisis aumenta.

5.2. Diseño del pipeline de análisis conductual

El pipeline recibe un video del FST y produce como salida los tiempos totales de cada conducta por animal y, de forma opcional, el desglose por minuto. Tiene cuatro etapas secuenciales: preprocesamiento, detección de cilindros, seguimiento de animales y clasificación de conducta.

Figura 5.2: Diagrama de flujo del pipeline de análisis conductual [autoría propia].

5.2.1. Preprocesamiento

El preprocesamiento decodifica el video y verifica que sea procesable. Si el contraste promedio de los cuadros cae por debajo del umbral mínimo, se aplica corrección CLAHE

(*Contrast Limited Adaptive Histogram Equalization*) [26] antes de continuar. Si después de CLAHE el contraste sigue siendo insuficiente, el pipeline se detiene y genera un reporte de diagnóstico (ver RN-11).

5.2.2. Detección de cilindros

El sistema localiza los cuatro cilindros en el encuadre usando detección de contornos sobre los primeros cuadros del video, después de construir el modelo de fondo por mediana. Cada cilindro detectado se convierte en una región de interés (ROI) que delimita el área de análisis de cada animal. La detección requiere una confianza mínima de 0.70; si no se alcanza para los cuatro cilindros, el pipeline se detiene.

5.2.3. Seguimiento de animales

Dentro de cada ROI, el seguimiento opera en tres niveles de respaldo:

1. **Segmentación por sustracción de fondo**: en cada cuadro se calcula la diferencia absoluta con el modelo de fondo y se umbraliza con Otsu [27]. Si el contorno resultante es consistente con el cuadro anterior (IoU $\geq$ umbral), se acepta como detección válida.

2. **Tracker CSRT**: si la segmentación falla o produce un resultado inconsistente, el tracker CSRT [21] mantiene la estimación de posición usando la apariencia del animal en cuadros anteriores.

3. **Congelación de posición**: si CSRT también falla, la última posición conocida se conserva con una confianza decreciente hasta que el detector vuelva a encontrar al animal.

La estimación de la línea de agua se estabiliza con un promedio móvil exponencial (EMA, ecuación 3.4 del cap. 3) para reducir el efecto de los reflejos en la detección de la rata.

5.2.4. Clasificación de conducta

El clasificador recibe el recorte de imagen alrededor de la rata (el bounding box), normalizado a 224×224 píxeles. Produce tres valores de probabilidad —uno por conducta—

y asigna la conducta con mayor valor. La arquitectura base es ResNet-18 preentrenada en ImageNet con transfer learning [24], [25]: se reemplaza la capa de clasificación original por una nueva con 3 salidas (nado activo, inmovilidad, escalamiento) y se re-entrena esa capa con los cuadros etiquetados del laboratorio.

La arquitectura del clasificador se detalla en la Figura 5.3.

Figura 5.3: Arquitectura del clasificador de conducta basado en ResNet-18 con transfer learning [autoría propia].

5.2.5. Estrategia de separación por sujeto

Los cuatro cilindros se graban en una sola toma cenital. La separación por sujeto se hace asignando cada ROI detectada en la etapa de detección de cilindros a un identificador de animal (Animal 1 a Animal 4). La asignación usa la posición espacial de cada ROI en el encuadre: se ordena primero por fila (arriba/abajo) y luego por columna (izquierda/derecha), lo que produce una numeración consistente entre sesiones mientras el encuadre no cambie.

Si el número de cilindros detectados es menor a cuatro, el sistema reporta cuántos animales pudo procesar y cuáles no, en lugar de abortar el análisis completo.

5.3. Diseño de la base de datos

El modelo de datos sigue el paradigma relacional. La Figura 5.4 muestra el diagrama entidad-relación físico con las tablas, columnas y claves del sistema.

Figura 5.4: Modelo entidad-relación físico del sistema [autoría propia].

Las tablas principales y sus responsabilidades son:

- **usuarios**: almacena credenciales (correo, hash de contraseña con bcrypt), rol (investigador/administrador) y estado (activo/inactivo).
- **experimentos**: agrupa los videos y resultados de un experimento. Campos: nombre, fecha, tratamiento, notas, número esperado de animales y referencia al usuario propietario.

- **videos**: un experimento puede tener hasta dos videos (Día 1 y Día 2). Campos: ruta del archivo, sesión (día 1/día 2), estado del análisis (en cola, procesando, completado, error), fecha de carga y fecha límite de eliminación (30 días desde la finalización del análisis).
- **animales**: cada video procesado genera un registro por animal detectado, con su identificador y posición en el encuadre.
- **resultados_conducta**: almacena el tiempo en segundos de cada conducta por animal, sesión y, opcionalmente, por minuto dentro de la sesión. Es la tabla central para la generación de reportes.
- **reportes**: registros de los archivos PDF y CSV generados, con su ruta de almacenamiento y fecha de generación. Los reportes se conservan aunque el video original haya sido eliminado (ver RN-06).
- **notificaciones**: avisos al investigador sobre eventos del sistema (análisis completado, error en análisis, video próximo a eliminarse).

El diccionario de datos completo con tipos, restricciones y descripciones de cada columna se incluye en los anexos.

5.4. Diseño de la API REST

La API REST sigue las convenciones HTTP estándar [30]. Todos los endpoints requieren autenticación con JWT salvo los de inicio de sesión y recuperación de contraseña. Las respuestas usan JSON con una estructura consistente: campo `data` para el resultado, `error` para mensajes de error y `status` para el código HTTP.

La Tabla 5.1 describe los endpoints principales agrupados por recurso.

Cuadro 5.1: Endpoints principales de la API REST.

Método	**Ruta**	**Descripción**	**Acceso**
POST	`/auth/login`	Inicia sesión; devuelve token JWT.	Público
POST	`/auth/logout`	Invalida el token activo.	Autenticado

Método	**Ruta**	**Descripción**	**Acceso**
POST	`/auth/recover`	Envía enlace de recuperación por correo.	Público
GET	`/experiments`	Lista los experimentos del usuario activo.	Investigador
POST	`/experiments`	Crea un nuevo experimento con sus metadatos.	Investigador
GET	`/experiments/{id}`	Devuelve detalle y estado de un experimento.	Investigador
DELETE	`/experiments/{id}`	Elimina permanentemente un experimento.	Investigador
POST	`/experiments/{id}/videos`	Sube un video (Día 1 o Día 2) y encola el análisis.	Investigador
GET	`/experiments/{id}/status`	Devuelve el progreso del análisis en porcentaje.	Investigador
GET	`/experiments/{id}/results`	Devuelve los resultados por animal y sesión.	Investigador
GET	`/experiments/{id}/results/by-minute`	Devuelve el desglose por minuto.	Investigador
GET	`/experiments/{id}/comparison`	Devuelve la comparación Día 1 vs. Día 2.	Investigador
GET	`/experiments/{id}/report/pdf`	Descarga el reporte en PDF.	Investigador
GET	`/experiments/{id}/report/csv`	Descarga los datos en CSV.	Investigador
GET	`/admin/users`	Lista todos los usuarios (filtrable por rol y estado).	Admin

Método	Ruta	Descripción	Acceso
POST	`/admin/users`	Crea una cuenta de usuario.	Admin
PATCH	`/admin/users/{id}`	Modifica o desactiva una cuenta.	Admin
GET	`/admin/system`	Devuelve métricas del sistema: disco y cola.	Admin

5.5. Diseño de casos de uso

El sistema tiene dos actores principales: **Investigador** y **Administrador**. El Investigador gestiona sus experimentos y consulta resultados; el Administrador gestiona usuarios y monitorea el estado del sistema. El Administrador hereda todas las capacidades del Investigador sobre los experimentos de cualquier usuario (ver RN-08).

Figura 5.5: Diagrama de casos de uso del sistema [autoría propia].

Los casos de uso principales son:

- **CU-01 Iniciar sesión:** el investigador ingresa correo y contraseña. El sistema valida las credenciales y devuelve un token JWT. Las credenciales incorrectas muestran un error genérico (ver RN-01).
- **CU-02 Crear experimento:** el investigador registra los metadatos del experimento (nombre, fecha, tratamiento, número de animales) y sube uno o dos videos. El análisis inicia automáticamente al completarse la carga válida.
- **CU-03 Consultar progreso:** el investigador puede ver el estado del análisis en tiempo real. Si ocurre un error, puede descargar el reporte de diagnóstico.
- **CU-04 Ver resultados:** una vez completado el análisis, el investigador puede consultar los tiempos por animal y conducta, el desglose por minuto y, si ambos videos fueron procesados, la comparación Día 1 vs. Día 2.
- **CU-05 Descargar reporte:** el investigador descarga los resultados en PDF o CSV desde la pantalla de resultados o desde el dashboard.

- **CU-06 Gestionar usuarios (Admin):** el administrador crea, modifica y desactiva cuentas. Las contraseñas temporales se marcan para cambio obligatorio al primer ingreso.
- **CU-07 Monitorear sistema (Admin):** el administrador consulta el uso de disco, el tamaño de la cola y las métricas operativas del sistema.

5.6. Diagramas de secuencia

Los diagramas de secuencia describen el flujo de mensajes entre el frontend, el backend y el worker para los flujos más importantes del sistema.

Figura 5.6: Secuencia: inicio de sesión [autoría propia].

Figura 5.7: Secuencia: carga de video y análisis automático [autoría propia].

Figura 5.8: Secuencia: manejo de error en el análisis [autoría propia].

5.7. Diagrama de clases

Las clases principales del dominio son:

- **User**: representa a un usuario del sistema con su rol y estado. Tiene métodos para verificar contraseña, generar token y cambiar estado.
- **Experiment**: agrupa los videos y resultados de un experimento. Conoce su estado general (en proceso, completado, error) derivado del estado de sus videos.
- **Video**: representa un archivo de video subido. Gestiona su ciclo de vida: validación, encolado, procesamiento y eliminación automática a 30 días.
- **Animal**: un animal detectado en un video. Tiene su identificador, posición en el encuadre y sus resultados de conducta asociados.

Figura 5.9: Diagrama de clases del sistema [autoría propia].

- **BehaviorResult**: tiempo en segundos de cada conducta para un animal en una sesión. Puede tener desglose por minuto.
- **Report**: archivo PDF o CSV generado para un experimento. Persiste aunque el video original haya sido eliminado.

5.8. Diseño de la interfaz web

La interfaz tiene seis pantallas principales, diseñadas para que un investigador sin experiencia en programación pueda completar el flujo completo —crear experimento, esperar el análisis, consultar y descargar resultados— sin necesitar asistencia técnica (ver RNF-06).

5.8.1. Pantalla de inicio de sesión

Formulario con campos de correo institucional y contraseña. Las credenciales incorrectas muestran un mensaje de error genérico sin revelar cuál campo falló (ver RN-01). Incluye enlace de recuperación de contraseña (RF-03).

Figura 5.10: Pantalla de inicio de sesión [autoría propia].

5.8.2. Dashboard del investigador

Lista de experimentos del investigador activo con nombre, fecha, tratamiento, sesión, estado y accesos directos a resultados y descarga. Muestra avisos cuando un video está a menos de 7 días de ser eliminado automáticamente (RF-23). Incluye filtros por estado y paginación.

Figura 5.11: Dashboard del investigador [autoría propia].

5.8.3. Creación de experimento

Formulario en tres pasos: metadatos del experimento, carga de videos (Día 1 y Día 2, ambos opcionales) y confirmación. Solo acepta archivos `.mp4`; el rechazo de formato ocurre

en el cliente antes de iniciar la subida. Una vez completada la carga válida, el análisis inicia automáticamente (RF-12).

La división en tres pasos se eligió sobre un formulario único porque la carga de video puede tardar varios minutos según el tamaño del archivo: separar los metadatos de la carga permite que el investigador corrija un nombre o fecha sin necesidad de volver a subir el video si cometió un error en los campos de texto.

Figura 5.12: Pantalla de creación de experimento [autoría propia].

5.8.4. Progreso del análisis

Muestra el estado actual del pipeline con las cuatro etapas (preprocesamiento, detección de cilindros, seguimiento y clasificación), una barra de progreso en porcentaje y el log de ejecución expandible. Si ocurre un error, muestra el código, la descripción y el botón de descarga del reporte de diagnóstico (RF-16). El análisis continúa aunque el investigador cierre la pestaña (RF-15).

No hay botón de cancelar visible durante el análisis. La decisión fue deliberada: el laboratorio no tiene casos de uso donde cancele un análisis a mitad de camino, y agregar la funcionalidad implicaría manejar el estado de cancelación en el worker y limpiar los archivos parciales. Si en el futuro surge esa necesidad, el endpoint `DELETE /experiments/{id}/videos` puede extenderse para eso.

Figura 5.13: Pantalla de progreso del análisis [autoría propia].

5.8.5. Resultados del experimento

Cuatro pestañas: resumen por sesión con totales en segundos y porcentaje por conducta, tabla detallada por animal (RF-18), desglose por minuto (RF-19) y comparación Día 1 vs. Día 2 (RF-20, disponible solo si ambos videos fueron procesados). Incluye botones de descarga en PDF y CSV.

Figura 5.14: Pantalla de resultados del experimento [autoría propia].

5.8.6. Panel de administración

Vista exclusiva del rol Administrador. Muestra métricas del sistema (uso de disco, cola de procesamiento, análisis activos), lista de usuarios con filtros por rol y estado, y la lista de experimentos de todos los investigadores. Incluye modales para crear y editar cuentas (RF-05, RF-27, RF-28).

Figura 5.15: Panel de administración [autoría propia].

Bibliografía

[1] A. Armario, «The forced swim test: Historical, conceptual and methodological considerations and its relationship with individual behavioral traits,» *Neuroscience and Biobehavioral Reviews*, vol. 128, págs. 74-86, 2021. DOI: 10.1016/j.neubiorev.2021.06.014.

[2] A. Della Valle, S. De Carlo, G. Sonsini, M. Ubaldi y R. Ciccocioppo, «Machine learning-based model for behavioural analysis in rodents applied to the forced swim test,» *Scientific Reports*, vol. 15, 2025. DOI: 10.1038/s41598-025-05712-8.

[3] World Health Organization, *Trastorno depresivo (depresión)*, 2025. dirección: https://www.who.int/es/news-room/fact-sheets/detail/depression (visitado 01-02-2026).

[4] R. D. Porsolt, A. Bertin y M. Jalfre, «Behavioral despair in mice: A primary screening test for antidepressants,» *Archives Internationales de Pharmacodynamie et de Therapie*, vol. 229, n.º 2, págs. 327-336, 1977.

[5] D. A. Slattery y J. F. Cryan, «Using the rat forced swim test to assess antidepressant-like activity in rodents,» *Nature Protocols*, vol. 7, n.º 6, págs. 1009-1014, 2012. DOI: 10.1038/nprot.2012.044.

[6] J. F. Cryan, C. Mombereau y A. Vassout, «The tail suspension test as a model for assessing antidepressant activity,» *Neuroscience and Biobehavioral Reviews*, vol. 29, págs. 571-625, 2005. DOI: 10.1016/j.neubiorev.2005.03.009.

[7] Secretaría de Agricultura, Ganadería, Desarrollo Rural, Pesca y Alimentación, «Norma Oficial Mexicana NOM-062-ZOO-1999: Especificaciones técnicas para la producción, cuidado y uso de los animales de laboratorio,» Diario Oficial de la Federación, inf. téc., 2001. dirección: https://www.gob.mx/cms/uploads/attachment/file/203594/NOM-062-ZOO-1999_220801.pdf (visitado 01-02-2026).

[8] Noldus Information Technology, *EthoVision XT | Video tracking software*, n.d. dirección: https://noldus.com/ethovision-xt (visitado 01-02-2026).

[9] Z. T. Pennington, Z. Dong, Y. Feng et al., «ezTrack: An open-source video analysis pipeline for the investigation of animal behavior,» *Scientific Reports*, vol. 9, n.º 1, pág. 19 979, 2019. DOI: 10.1038/s41598-019-56408-9.

[10] A. Nandi, G. Virmani, A. Barve y S. Marathe, «DBscorer: An Open-Source Software for Automated Accurate Analysis of Rodent Behavior in Forced Swim Test and Tail Suspension Test,» *eNeuro*, vol. 8, n.º 6, 2021. DOI: 10.1523/ENEURO.0305-21.2021.

[11] X. Meng, Y. Xia, M. Liu et al., «A deep-learning-based threshold-free method for automated analysis of rodent behavior in the forced swim test and tail suspension test,» *Journal of Neuroscience Methods*, vol. 409, pág. 110 212, 2024. DOI: 10.1016/j.jneumeth.2024.110212.

[12] ISO/IEC, «ISO/IEC 25010:2023 — Systems and software engineering — Systems and software quality models,» International Organization for Standardization, inf. téc., 2023.

[13] ISO/IEC/IEEE, «ISO/IEC/IEEE 12207:2017 — Systems and software engineering — Software life cycle processes,» International Organization for Standardization, inf. téc., 2017.

[14] O. Sturman, L. von Ziegler, C. Schlaeppi et al., «Deep learning-based behavioral analysis enables high precision rodent tracking and is capable of outperforming commercial solutions,» *Neuropsychopharmacology*, vol. 45, págs. 1137-1144, 2020. DOI: 10.1038/s41386-020-0776-y.

[15] Y.-H. Chan, J. A. Klarevas-Irby y D. R. Farine, «YOLO-Behaviour: A simple, flexible framework to automatically quantify animal behaviours from videos,» *Methods in Ecology and Evolution*, 2025. DOI: 10.1111/2041-210X.14502.

[16] Z. Zivkovic, «Improved adaptive Gaussian mixture model for background subtraction,» en *Proceedings of the 17th International Conference on Pattern Recognition (ICPR)*, vol. 2, 2004, págs. 28-31. DOI: 10.1109/ICPR.2004.1333992.

[17] M. Piccardi, «Background subtraction techniques: a review,» en *Proceedings of the IEEE International Conference on Systems, Man and Cybernetics*, 2004, págs. 3099-3104. DOI: 10.1109/ICSMC.2004.1400815.

[18] G. Jocher, A. Chaurasia y J. Qiu, *Ultralytics YOLO (Version 8.0.0)*, 2023. dirección: https://github.com/ultralytics/ultralytics (visitado 01-02-2026).

[19] J. H. Stadler, H. Luckow y C. Goeke, «Automatic object detection for behavioural research using YOLOv8,» *Behavior Research Methods*, vol. 56, n.º 7, págs. 7307-7330, 2024. DOI: 10.3758/s13428-024-02420-5.

[20] A. Mathis, P. Mamidanna, K. M. Cury et al., «DeepLabCut: Markerless pose estimation of user-defined body parts with deep learning,» *Nature Neuroscience*, vol. 21, n.º 9, págs. 1281-1289, 2018. DOI: 10.1038/s41593-018-0209-y.

[21] A. Lukezic, T. Vojir, L. Cehovin, J. Matas y M. Kristan, «Discriminative Correlation Filter with Channel and Spatial Reliability,» en *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, págs. 4847-4856. DOI: 10.1109/CVPR.2017.515.

[22] J. F. Henriques, R. Caseiro, P. Martins y J. Batista, «High-Speed Tracking with Kernelized Correlation Filters,» *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 37, n.º 3, págs. 583-596, 2015. DOI: 10.1109/TPAMI.2014.2345390.

[23] Y. LeCun, L. Bottou, Y. Bengio y P. Haffner, «Gradient-based learning applied to document recognition,» *Proceedings of the IEEE*, vol. 86, n.º 11, págs. 2278-2324, 1998. DOI: 10.1109/5.726791.

[24] S. J. Pan y Q. Yang, «A survey on transfer learning,» *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, n.º 10, págs. 1345-1359, 2010. DOI: 10.1109/TKDE.2009.191.

[25] K. He, X. Zhang, S. Ren y J. Sun, «Deep Residual Learning for Image Recognition,» en *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, págs. 770-778. DOI: 10.1109/CVPR.2016.90.

[26] G. Bradski y A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media, 2008.

[27] N. Otsu, «A threshold selection method from gray-level histograms,» *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, n.º 1, págs. 62-66, 1979. DOI: 10.1109/TSMC.1979.4310076.

[28] S. Suzuki y K. Abe, «Topological structural analysis of digitized binary images by border following,» *Computer Vision, Graphics, and Image Processing*, vol. 30, n.º 1, págs. 32-46, 1985. DOI: 10.1016/0734-189X(85)90016-7.

[29] D. P. Kingma y J. Ba, «Adam: A Method for Stochastic Optimization,» en *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015. dirección: `https://arxiv.org/abs/1412.6980`.

[30] M. Grinberg, *Flask Web Development: Developing Web Applications with Python*, 2.ª ed. O'Reilly Media, 2018.

[31] D. Merkel, «Docker: Lightweight Linux Containers for Consistent Development and Deployment,» *Linux Journal*, vol. 2014, n.º 239, pág. 2, 2014.

[32] R. S. Pressman y B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8.ª ed. McGraw-Hill Education, 2014.

[33] I. Sommerville, *Software Engineering*, 10.ª ed. Pearson, 2016.

[34] M. Abadi, P. Barham, J. Chen et al., «TensorFlow: A System for Large-Scale Machine Learning,» en *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2016, págs. 265-283. dirección: `https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi`.

[35] F. Pedregosa, G. Varoquaux, A. Gramfort et al., «Scikit-learn: Machine Learning in Python,» *Journal of Machine Learning Research*, vol. 12, págs. 2825-2830, 2011.

[36] O. Friard y M. Gamba, «BORIS: a free, versatile open-source event-logging software for video/audio coding and live observations,» *Methods in Ecology and Evolution*, vol. 7, n.º 11, págs. 1325-1330, 2016. DOI: `10.1111/2041-210X.12584`.